Beside 백엔드 성능 보고서

측정일 2026-10-31 · 반복 v0 3회, v1 3회, v1p 3회, v2 3회, v2p 3회 · 기준 v1

> ⚠ **다른 프로그램이 돌던 PC**: 스위트 직전 CPU 사용률 중앙 19 %, 최대 33 % (측정 방법의 '한계'). 버전 순서를 반복마다 돌려 한 버전에만 몰리지 않게 했고, 판정 규칙이 반복 사이 흔들림을 걸러 낸다.

버전

라벨	제목	커밋	설명
v0	이전 (10-14, #1 머지)	`e0e1c2b34` e0e1c2b34b1b94669d824fd0fd4afeea9fb800f1	origin/main. Unity 앱·AnimalLift 연동 직후. 업로드 전송·prod 프로파일 없음.
v1 (기준)	현재 (4b 배포 기반)	`af4a747b1` af4a747b1c36f2380d1ff69a523b5a9dbaca7503	main = deploy/vm-4b. prod 프로파일, /infer 업로드 전송, GPU 감시·오프라인 대기, Discord 알림, H2 백업.
v1p	현재 + 개선	`033f3f371` 033f3f3719745e3d4776c5ea6e4e2e7db536f027	perf/backend-report. v1 + P1(작업 목록 키셋 페이지, jobs(created_at, id) 인덱스).
v2	다음 버전 후보 (v1.1)	`6a3959f60` 6a3959f60380947ad2f430be24f5d50661a57c95	review/api-v1.1 의 rebase 전(백업 ref backup/review-api-v1.1-6a3959f). v1 + 관리 화면·대기 순번·취소·공유·평가·보관함·썸네일. 계약 회의 전.
v2p	다음 버전 + 개선	`ecc5fb5f9` ecc5fb5f9ba0bee443137ce8c59a5223174e7dc2	v2 를 v1p 위로 rebase + P4(썸네일을 202 뒤 백그라운드로) + P5(jobs·job_runs 인덱스, 대기 순번 범위 쿼리).

핵심 지표

칸: p50 / p95 (ms) 또는 값, 아래 줄은 기준 대비 p50 변화율과 판정. 판정은 |Δ| ≥ 10% 이고 반복별 p50 범위가 기준과 겹치지 않을 때만 개선/악화, 나머지는 차이 없음(잡음). 처리량은 클수록 개선. 기준이 0 이면 비교 불가.

지표	v0	v1	v1p	v2	v2p
기동 시간 `bench.startupMs` · ms · 값	7,874 3회 +1.8% = 차이 없음	7,738 3회 기준	7,999 3회 +3.4% = 차이 없음	8,828 3회 +14.1% ! 악화	8,647 3회 +11.7% ! 악화
유휴 메모리 (RSS) `bench.idleRssMB` · MiB · 값	354.5 3회 −1.0% = 차이 없음	358.2 3회 기준	361.2 3회 +0.8% = 차이 없음	377.8 3회 +5.5% = 차이 없음	371.9 3회 +3.8% = 차이 없음
healthz 응답 `healthzMs` · ms · p50 / p95	0.52 / 2.23 n=1500 −1.9% = 차이 없음	0.53 / 2.33 n=1500 기준	0.51 / 2.54 n=1500 −3.8% = 차이 없음	0.52 / 3.02 n=1500 −1.9% = 차이 없음	0.53 / 2.71 n=1500 ±0.0% = 차이 없음
작은 사진 업로드 `postMs` · ms · p50 / p95	12.7 / 32.7 n=300 −3.9% = 차이 없음	13.2 / 37.0 n=300 기준	13.3 / 37.3 n=300 +0.5% = 차이 없음	19.8 / 35.8 n=300 +50.3% ! 악화	16.9 / 48.5 n=300 +28.2% ! 악화
큰 사진 4장 업로드 `postMs` · ms · p50 / p95	132 / 161 n=60 −3.5% = 차이 없음	136 / 158 n=60 기준	136 / 162 n=60 −0.1% = 차이 없음	366 / 379 n=60 +168.1% ! 악화	139 / 162 n=60 +1.8% = 차이 없음
12 MP 사진 업로드 `postMs` · ms · p50 / p95	15.8 / 30.7 n=60 +5.1% = 차이 없음	15.0 / 18.9 n=60 기준	15.3 / 19.7 n=60 +2.1% = 차이 없음	70.1 / 81.6 n=60 +366.5% ! 악화	15.5 / 24.3 n=60 +3.1% = 차이 없음
12 MP 동시 업로드 `postConcurrentMs` · ms · p50 / p95	62.5 / 160 n=60 +5.9% = 차이 없음	59.1 / 113 n=60 기준	54.9 / 120 n=60 −7.1% = 차이 없음	235 / 279 n=60 +297.1% ! 악화	54.5 / 160 n=60 −7.7% = 차이 없음
큰 사진 업로드의 서버 CPU 점유 `bench.serverCpuMs` · ms · 값	3,641 3회 −12.1%	4,141 3회 기준	3,922 3회 −5.3%	4,594 3회 +10.9%	4,922 3회 +18.9%
E2E 흐름 (mock) `flowMs` · ms · p50 / p95	25.8 / 49.4 n=150 +36.8% ! 악화	18.9 / 37.5 n=150 기준	23.5 / 40.0 n=150 +24.3% ! 악화	20.7 / 31.9 n=150 +9.5% = 차이 없음	30.2 / 46.3 n=150 +60.0% ! 악화

지표	v0	v1	v1p	v2	v2p
동시 E2E 흐름 `flowMs` · ms · p50 / p95	219 / 468 n=480 +12.1% = 차이 없음	195 / 474 n=480 기준	189 / 440 n=480 −3.1% = 차이 없음	319 / 518 n=480 +63.6% ! 악화	280 / 584 n=480 +43.6% ! 악화
동시 E2E 처리량 `bench.jobsPerSec` · 건/s · 값	64.9 3회 +4.4% = 차이 없음	62.2 3회 기준	67.0 3회 +7.8% = 차이 없음	47.2 3회 −24.0% ! 악화	47.1 3회 −24.3% ! 악화
폴링 부하 응답 `pollMs` · ms · p50 / p95	6.36 / 19.2 n=12000 −0.2% = 차이 없음	6.37 / 17.4 n=12000 기준	6.35 / 21.6 n=12000 −0.3% = 차이 없음	8.59 / 30.7 n=12000 +34.9% ! 악화	7.74 / 29.3 n=12000 +21.4% = 차이 없음
폴링 처리량 `bench.pollReqPerSec` · 요청/s · 값	1,797.8 3회 +2.6% = 차이 없음	1,752.4 3회 기준	1,723.7 3회 −1.6% = 차이 없음	1,291.2 3회 −26.3% ! 악화	1,320.4 3회 −24.6% ! 악화
목록 첫 페이지 (N=2000) `listFirstMs@2000` · ms · p50 / p95	13.4 / 20.7 n=300 +2.6% = 차이 없음	13.1 / 20.0 n=300 기준	3.64 / 4.65 n=300 −72.2% ✓ 개선	14.7 / 21.3 n=300 +12.6% = 차이 없음	6.78 / 8.54 n=300 −48.2% ✓ 개선
목록 중간 페이지 (N=2000) `listDeepMs@2000` · ms · p50 / p95	23.0 / 31.9 n=150 +4.1% = 차이 없음	22.1 / 29.8 n=150 기준	15.0 / 18.2 n=150 −32.3% ✓ 개선	33.8 / 39.5 n=150 +52.7% ! 악화	25.2 / 31.1 n=150 +14.1% ! 악화
단건 조회 (N=2000) `getMs@2000` · ms · p50 / p95	0.50 / 0.88 n=600 ±0.0% = 차이 없음	0.50 / 0.66 n=600 기준	0.49 / 0.78 n=600 −2.0% = 차이 없음	0.63 / 0.79 n=600 +26.0% ! 악화	0.65 / 0.85 n=600 +30.0% ! 악화
업로드 (N=2000) `postMs@2000` · ms · p50 / p95	5.82 / 8.55 n=90 +13.8% = 차이 없음	5.11 / 8.02 n=90 기준	5.66 / 7.64 n=90 +10.7% = 차이 없음	9.68 / 11.3 n=90 +89.3% ! 악화	4.92 / 8.71 n=90 −3.7% = 차이 없음
재기동 시간 `bench.restartStartupMs` · ms · 값	7,725 3회 −2.1% = 차이 없음	7,888 3회 기준	8,204 3회 +4.0% = 차이 없음	9,214 3회 +16.8% ! 악화	8,937 3회 +13.3% ! 악화
PENDING 조회 `pendingPollMs` · ms · p50 / p95	2.42 / 6.46 n=600 −6.7% = 차이 없음	2.60 / 6.65 n=600 기준	2.33 / 6.63 n=600 −10.6% = 차이 없음	4.89 / 12.7 n=600 +88.3% ! 악화	4.66 / 14.0 n=600 +79.2% ! 악화
최대 메모리 (RSS) `bench.peakRssMB` · MiB · 값	494.9 3회 −0.8% = 차이 없음	499.1 3회 기준	448.6 3회 −10.1% ✓ 개선	502.9 3회 +0.8% = 차이 없음	507.3 3회 +1.7% = 차이 없음
real · path 흐름 `flowMs` · ms · p50 / p95	81.6 / 98.4 n=60 +0.4% = 차이 없음	81.2 / 95.5 n=60 기준	81.3 / 97.5 n=60 ±0.0% = 차이 없음	285 / 313 n=60 +250.6% ! 악화	86.9 / 112 n=60 +7.0% = 차이 없음
real · upload 흐름 `flowMs` · ms · p50 / p95	측정 안 함 이 버전에 없는 기능 (real_upload)	326 / 369 n=60 기준	327 / 357 n=60 +0.5% = 차이 없음	330 / 371 n=60 +1.4% = 차이 없음	331 / 364 n=60 +1.5% = 차이 없음
real,prod · upload 흐름 `flowMs` · ms · p50 / p95	측정 안 함 이 버전에 없는 기능 (real_upload, prod)	330 / 358 n=60 기준	328 / 355 n=60 −0.7% = 차이 없음	328 / 378 n=60 −0.6% = 차이 없음	335 / 376 n=60 +1.5% = 차이 없음
작업 사이 빈 시간 (prod) `queueGapMs` · ms · p50 / p95	측정 안 함 이 버전에 없는 기능 (real_upload, prod)	17.8 / 23.6 n=12 기준	16.2 / 25.5 n=12 −8.9% = 차이 없음	17.6 / 23.1 n=12 −1.3% = 차이 없음	23.2 / 45.4 n=12 +30.5% = 차이 없음

쌍 비교

versions 파일의 `pairs`: 열마다 앞 버전을 기준으로 뒤 버전을 비교한다. 칸: 앞 → 뒤의 p50(값 지표는 반복별 값의 중앙값), 아래 줄은 변화율과 판정(위 표와 같은 규칙).

지표	이전 → 현재 v0 → v1	현재의 개선 (P1) v1 → v1p	다음 버전 기능의 비용 v1 → v2	다음 버전의 개선 (P1·P4·P5) v2 → v2p
기동 시간 `bench.startupMs` · ms	7,874 → 7,738 −1.7% = 차이 없음	7,738 → 7,999 +3.4% = 차이 없음	7,738 → 8,828 +14.1% ! 악화	8,828 → 8,647 −2.0% = 차이 없음
유휴 메모리 (RSS) `bench.idleRssMB` · MiB	354.5 → 358.2 +1.0% = 차이 없음	358.2 → 361.2 +0.8% = 차이 없음	358.2 → 377.8 +5.5% = 차이 없음	377.8 → 371.9 −1.5% = 차이 없음
healthz 응답 `healthzMs` · ms	0.52 → 0.53 +1.9% = 차이 없음	0.53 → 0.51 −3.8% = 차이 없음	0.53 → 0.52 −1.9% = 차이 없음	0.52 → 0.53 +1.9% = 차이 없음
작은 사진 업로드 `postMs` · ms	12.7 → 13.2 +4.1% = 차이 없음	13.2 → 13.3 +0.5% = 차이 없음	13.2 → 19.8 +50.3% ! 악화	19.8 → 16.9 −14.7% ✓ 개선
큰 사진 4장 업로드 `postMs` · ms	132 → 136 +3.7% = 차이 없음	136 → 136 −0.1% = 차이 없음	136 → 366 +168.1% ! 악화	366 → 139 −62.0% ✓ 개선
12 MP 사진 업로드 `postMs` · ms	15.8 → 15.0 −4.9% = 차이 없음	15.0 → 15.3 +2.1% = 차이 없음	15.0 → 70.1 +366.5% ! 악화	70.1 → 15.5 −77.9% ✓ 개선
12 MP 동시 업로드 `postConcurrentMs` · ms	62.5 → 59.1 −5.5% = 차이 없음	59.1 → 54.9 −7.1% = 차이 없음	59.1 → 235 +297.1% ! 악화	235 → 54.5 −76.8% ✓ 개선
큰 사진 업로드의 서버 CPU 점유 `bench.serverCpuMs` · ms	3,641 → 4,141 +13.7%	4,141 → 3,922 −5.3%	4,141 → 4,594 +10.9%	4,594 → 4,922 +7.1%
E2E 흐름 (mock) `flowMs` · ms	25.8 → 18.9 −26.9% ✓ 개선	18.9 → 23.5 +24.3% ! 악화	18.9 → 20.7 +9.5% = 차이 없음	20.7 → 30.2 +46.1% ! 악화
동시 E2E 흐름 `flowMs` · ms	219 → 195 −10.8% = 차이 없음	195 → 189 −3.1% = 차이 없음	195 → 319 +63.6% ! 악화	319 → 280 −12.2% ✓ 개선
동시 E2E 처리량 `bench.jobsPerSec` · 건/s	64.9 → 62.2 −4.2% = 차이 없음	62.2 → 67.0 +7.8% = 차이 없음	62.2 → 47.2 −24.0% ! 악화	47.2 → 47.1 −0.3% = 차이 없음
폴링 부하 응답 `pollMs` · ms	6.36 → 6.37 +0.2% = 차이 없음	6.37 → 6.35 −0.3% = 차이 없음	6.37 → 8.59 +34.9% ! 악화	8.59 → 7.74 −9.9% = 차이 없음
폴링 처리량 `bench.pollReqPerSec` · 요청/s	1,797.8 → 1,752.4 −2.5% = 차이 없음	1,752.4 → 1,723.7 −1.6% = 차이 없음	1,752.4 → 1,291.2 −26.3% ! 악화	1,291.2 → 1,320.4 +2.3% = 차이 없음
목록 첫 페이지 (N=2000) `listFirstMs@2000` · ms	13.4 → 13.1 −2.5% = 차이 없음	13.1 → 3.64 −72.2% ✓ 개선	13.1 → 14.7 +12.6% = 차이 없음	14.7 → 6.78 −54.0% ✓ 개선
목록 중간 페이지 (N=2000) `listDeepMs@2000` · ms	23.0 → 22.1 −3.9% = 차이 없음	22.1 → 15.0 −32.3% ✓ 개선	22.1 → 33.8 +52.7% ! 악화	33.8 → 25.2 −25.3% ✓ 개선
단건 조회 (N=2000) `getMs@2000` · ms	0.50 → 0.50 ±0.0% = 차이 없음	0.50 → 0.49 −2.0% = 차이 없음	0.50 → 0.63 +26.0% ! 악화	0.63 → 0.65 +3.2% = 차이 없음
업로드 (N=2000) `postMs@2000` · ms	5.82 → 5.11 −12.1% = 차이 없음	5.11 → 5.66 +10.7% = 차이 없음	5.11 → 9.68 +89.3% ! 악화	9.68 → 4.92 −49.1% ✓ 개선
재기동 시간 `bench.restartStartupMs` · ms	7,725 → 7,888 +2.1% = 차이 없음	7,888 → 8,204 +4.0% = 차이 없음	7,888 → 9,214 +16.8% ! 악화	9,214 → 8,937 −3.0% = 차이 없음
PENDING 조회 `pendingPollMs` · ms	2.42 → 2.60 +7.2% = 차이 없음	2.60 → 2.33 −10.6% = 차이 없음	2.60 → 4.89 +88.3% ! 악화	4.89 → 4.66 −4.8% = 차이 없음
최대 메모리 (RSS) `bench.peakRssMB` · MiB	494.9 → 499.1 +0.8% = 차이 없음	499.1 → 448.6 −10.1% ✓ 개선	499.1 → 502.9 +0.8% = 차이 없음	502.9 → 507.3 +0.9% = 차이 없음
real · path 흐름 `flowMs` · ms	81.6 → 81.2 −0.4% = 차이 없음	81.2 → 81.3 ±0.0% = 차이 없음	81.2 → 285 +250.6% ! 악화	285 → 86.9 −69.5% ✓ 개선
real · upload 흐름 `flowMs` · ms	–	326 → 327 +0.5% = 차이 없음	326 → 330 +1.4% = 차이 없음	330 → 331 +0.1% = 차이 없음
real,prod · upload 흐름 `flowMs` · ms	–	330 → 328 −0.7% = 차이 없음	330 → 328 −0.6% = 차이 없음	328 → 335 +2.1% = 차이 없음
작업 사이 빈 시간 (prod) `queueGapMs` · ms	–	17.8 → 16.2 −8.9% = 차이 없음	17.8 → 17.6 −1.3% = 차이 없음	17.6 → 23.2 +32.3% ! 악화

측정 방법

환경

항목	값
OS	Windows-11-10.0.26300-SP0
CPU	AMD Ryzen 5 5600X 6-Core Processor
논리 CPU	12
메모리 (GB)	31.9
Java	openjdk version "17.0.20.1" 2026-08-18 LTS
Python (부하 생성기)	3.14.0
JVM 옵션	-Xms256m -Xmx1536m -XX:ActiveProcessorCount=2 -XX:+UseG1GC
서버 CPU 고정	10, 11 (마지막 논리 CPU, 물리 코어 1개)
전원	source ac, battery none, plan 고성능 (8c5e7fda-e8bf-4a96-9a85-a6e23a8c635c)

서버 조건

- JVM: `-Xms256m -Xmx1536m -XX:ActiveProcessorCount=2 -XX:+UseG1GC`, 버전마다 `bootJar` 로 만든 jar 를 `java -jar` 로 실행.
- 서버 프로세스는 마지막 논리 CPU 10, 11 에 고정했다(물리 코어 1개를 SMT 로 나눈 논리 CPU 2개라 계산 자원은 코어 1개 분량이다). JVM 에는 `-XX:ActiveProcessorCount=2` 로 CPU 2개라고 알렸다. 부하 생성기(Python)와 추론 스텁은 고정하지 않는다.
- 서버 A·B: 프로파일 `mock`, 지연 0 (`mock.worker.pending-ms=0`, `processing-ms=0`), 실패 트리거 끔, 결과 GLB 는 합성 GLB(1,500,000 바이트). 서버 B 는 A 의 저장소로 다시 띄우고 `pending-ms=600000` 으로 작업을 PENDING 에 둔다.
- 서버 C: 프로파일 `real` 또는 `real,prod`, 추론 서비스 스텁(`generation/inference_service/app.py`, 패스스루: 모델 없이 같은 GLB 를 즉시 돌려줌), 전송 path / upload, 토큰 없음.
- 저장소(H2 파일 DB, 업로드, 결과, events.jsonl)는 작업 폴더 `C:\Users\minwo\AppData\Local\Temp\beside-perf` 의 실행 폴더 아래에 서버 A 와 서버 C(real 시나리오마다)를 띄울 때마다 새로 만든다. 서버 B 는 A 의 저장소를 그대로 다시 연다(재기동 시나리오). 모든 `storage.*` 경로는 절대 경로로 지정.
- 클라이언트: Python `http.client`, 127.0.0.1 루프백, 스레드마다 keep-alive 연결. 시간은 요청 쓰기 직전부터 응답 본문을 다 읽을 때까지(`perf_counter_ns`), 연결 수립은 제외 — E2E 흐름(`flowMs`·`completeMs`)도 흐름 안에서 다시 연결한 시간(Tomcat 은 100번째 응답에 연결을 닫는다)을 뺀다. POST 마다 새 `Idempotency-Key`.
- 반복마다 버전 순서를 돌린다(1회차 v0,v1,v2 → 2회차 v1,v2,v0 → …). 워밍업 요청은 기록하지 않는다.
- 서버가 쉴 때까지 기다린다: 서버 프로세스가 한 창 동안 CPU 를 한 코어의 5 % 이하로 쓰면 쉬는 것으로 본다. 서버 A 의 시나리오가 끝날 때마다 1초 창(최대 120초), 큰 사진·12 MP 업로드와 real 흐름은 하나마다 시작 전에 0.5초 창(최대 30초), 12 MP 동시 업로드·scale 의 측정 전과 PENDING 작업을 만든 뒤에도 기다린다. 응답 뒤에 남은 일(백그라운드 썸네일 큐, GC, JIT)이 다음 측정에 섞이지 않게 하려는 것이다(실사용에서 업로드는 몇 분 간격). 쉬는 것을 확인한 창 하나를 뺀 대기는 `bench.settleAfterMs`·`bench.settleTotalMs`, 시나리오 시작부터 대기 끝까지 서버가 쓴 CPU 시간은 `bench.serverCpuMs` 다. CPU 시간은 Windows 에서 스케줄러 틱(약 15.6 ms) 단위이고 고정한 두 논리 CPU 가 한 물리 코어를 나눠 쓰므로 반복 사이 흔들림이 커서 변화율만 보이고 판정하지 않는다.
- 프로세스 우선순위: 서버·추론 스텁·부하 생성기를 '보통 이상'(Windows ABOVE_NORMAL)으로 띄웠다. 같은 PC 의 다른 프로그램이 서버를 고정한 CPU 를 차지해 서버가 몇 초씩 멈추는 일을 줄이려는 것이다(클라우드 VM 에는 다른 프로그램이 없다).

시나리오

이름	무엇을	횟수
서버 기동 `startup`	프로세스 시작(Popen) → 첫 healthz 200, Spring 'Started' 로그 시간, 그 3초 뒤 RSS. 서버 A, 빈 저장소.	서버당 1회

이름	무엇을	횟수
헬스 체크 `healthz`	GET /api/v1/healthz 순차, keep-alive 연결 하나.	워밍업 100 + 측정 500
작은 사진 업로드 `upload_small`	64×64 PNG(약 12 KB) 1장 POST /api/v1/jobs 순차.	워밍업 10 + 측정 100
큰 사진 업로드 `upload_large`	약 4.5 MB 노이즈 PNG 4장(요청 하나에 약 18 MB) POST 순차.	워밍업 2 + 측정 20, 요청당 4장
12 MP 사진 업로드 `upload_12mp`	4000×3000 그라디언트 PNG(약 1.1 MB, 디코딩 부담) 1장 POST 순차, 이어서 4스레드 동시.	워밍업 2 + 측정 20, 동시 4스레드 × 5
E2E 흐름 (mock) `e2e_mock`	POST → 10 ms 간격 폴링 → COMPLETED → GLB 다운로드(크기 확인), 순차.	워밍업 5 + 측정 50, 폴링 10 ms
동시 E2E 흐름 `concurrent_e2e`	16스레드가 각자 E2E 흐름을 반복, 폴링 간격 50 ms.	16스레드 × 10흐름, 폴링 50 ms
폴링 부하 `poll_load`	COMPLETED 작업 하나를 4프로세스 × 4스레드가 GET /api/v1/jobs/{id} 로 반복 조회.	4프로세스 × 4스레드 × 250 (스레드마다 연결용 워밍업 5)
작업 수에 따른 변화 `scale`	작업을 N 개까지 쌓은 뒤(8스레드 업로드) 목록 첫 페이지·중간 페이지, 단건 조회, 업로드.	N = 200, 1000, 2000; 목록 첫 100, 중간 50, 조회 200, 업로드 30 (각 워밍업 5, 업로드 제외)
자원 `resources`	mock 시나리오를 모두 마친 서버 A 의 최대 RSS(작업 집합 최대치)와 jar 크기.	1회
재기동·PENDING 폴링 `pending_poll`	서버 A 의 저장소로 재기동(서버 B, 작업이 PENDING 에 머묾): 기동 시간, PENDING 작업 조회.	PENDING 작업 5개, 워밍업 20 + 측정 200
real · path 전송 `real_path`	real 프로파일 + 추론 스텁(패스스루, 모델 없음), 사진 경로 전달. 4.5 MB PNG 1장 E2E 순차, 이어서 5건 동시 접수.	워밍업 3 + 측정 20, 폴링 20 ms, 동시 접수 5건
real · upload 전송 `real_upload`	real_path 와 같고 /infer/upload 로 사진을 올리고 GLB 를 내려받는다.	워밍업 3 + 측정 20, 폴링 20 ms, 동시 접수 5건
real,prod · upload 전송 `real_upload_prod`	real,prod 프로파일: 사진 시그니처 검사, 작업마다 추론 서비스 healthz 확인. upload 전송.	워밍업 3 + 측정 20, 폴링 20 ms, 동시 접수 5건

통계와 판정

- p50·p95 등 백분위수는 반복을 합친 표본에서 선형 보간(numpy 'linear', Hyndman–Fan type 7)으로 구한다. 원시 표본은 0.01 ms 단위로 저장한다.
- Δ% = (후보 p50 − 기준 p50) ÷ 기준 p50 × 100. 값 지표(기동 시간, RSS, 처리량)는 반복별 값의 중앙값끼리 비교한다.
- 판정: |Δ%| ≥ 10 이고 반복별 p50(값 지표는 반복별 값)의 [최소, 최대] 범위가 기준의 범위와 겹치지 않을 때만 **개선** 또는 **악화**. 그 밖에는 **차이 없음**(잡음으로 본다). 시간은 작을수록, 처리량은 클수록 개선. 기준이 0 이고 후보가 0 이 아니면 변화율이 없으므로 **비교 불가**.
- 오류 수(`bench.errors`)는 측정값이 아니라 사건 수라 반복을 **합한** 값을 보이고, 0 이 아니면 빨간색이며 판정하지 않는다. 실패한 요청이 하나라도 있으면 그 시나리오는 '오류 포함'으로 표시되고, 표본(p50·p95)은 성공한 요청만으로 계산된다.

이름 규칙

- `bench.*` 값과 클라이언트 표본 이름(`postMs`, `flowMs`, `pollMs`, `assetMs` 등)은 이 벤치마크 전용이며 docs/METRICS.md 에 등록된 이름이 아니다. 앱의 `uploadMs`(METRICS §3)와 섞지 않는다.
- 서버 값 `uploadMs`, `queuedMs`, `processingMs`, `totalMs` (표의 '· events' 행과 real 시나리오의 `server*` 표본)은 서버 events.jsonl 의 METRICS §1 정의 그대로다. real 시나리오는 스텁 패스스루라 `passthrough=true` 이고 모델 측정이 아니다.

한계

- 로컬 루프백(127.0.0.1)에서 잰 값이다. 클라우드 VM, Tailscale, 실제 GPU 추론 경로가 아니다.
- real 시나리오는 모델 없는 스텁이라 Spring 쪽 오버헤드(전송·저장·대기열)만 보인다.
- 부하 생성기는 Python(GIL, http.client)이라 동시 시나리오에는 클라이언트 자신의 지연이 섞일 수 있다.
- 서버와 클라이언트가 같은 PC 에서 돈다. 서버는 논리 CPU 10, 11(2개, 물리 코어 1개)에 고정했지만 부하 생성기와 추론 스텁은 고정하지 않았고, 디스크·메모리 대역은 공유한다. 백신 실시간 검사 등 OS 활동도 포함된다.
- 다른 프로그램의 부하: 스위트마다 시작 직전 2초 동안(이 도구의 서버는 꺼진 상태) 잰 PC 전체 CPU 사용률이 최소 17 %, 중앙 19 %, 최대 33 % 였다(스위트 15개). 서버를 고정한 CPU 도 다른 프로그램이 쓸 수 있어 이만큼 잡음이 섞인다.
- 갓 쓴 파일의 첫 열기가 느리다: 4.5 MB 사진 사본을 쓰자마자 열어 읽는 데 216 ms, 다시 읽으면 1.3 ms 였다(실시간 백신 검사로 보인다). 업로드 직후 사진을 여는 서버 코드(요청 안에서 만드는 썸네일, 추론 서비스로의 업로드 전송)는 이 대기를 포함하고, 그런 검사가 없는 Linux VM 에서는 이만큼 빠진다.
- 서버 쪽 시간(events.jsonl)은 ms 단위 정수(버림)라 1 ms 보다 작은 차이는 보이지 않는다 (이 측정의 서버: openjdk version "17.0.20.1" 2026-08-18 LTS · Windows-11-10.0.26300-SP0).

시나리오별 결과

표: p50 / p95 와 표본 수 n(값 지표는 반복의 중앙값), 아래 줄은 기준 대비 Δ% 와 판정. 그림: 같은 값을 막대로.

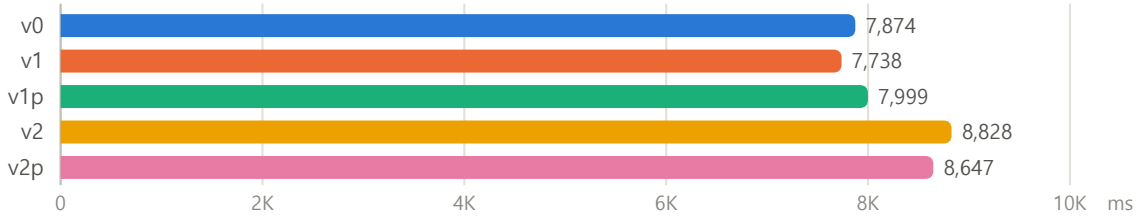
1. 서버 기동 `startup`

프로세스 시작(Popen) → 첫 healthz 200, Spring 'Started' 로그 시간, 그 3초 뒤 RSS. 서버 A, 빈 저장소. 서버당 1회.

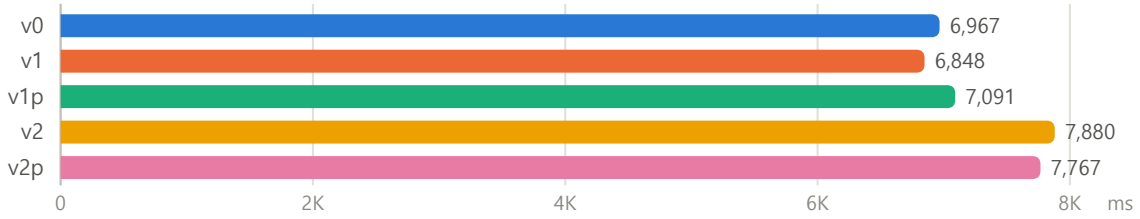
지표	v0	v1	v1p	v2	v2p
기동 (Popen→healthz 200) `bench.startupMs` · ms · 값	7,874 3회 +1.8% = 차이 없음	7,738 3회 기준	7,999 3회 +3.4% = 차이 없음	8,828 3회 +14.1% ! 악화	8,647 3회 +11.7% ! 악화
Spring 'Started' 로그 시간 `bench.springStartedMs` · ms · 값	6,967 3회 +1.7% = 차이 없음	6,848 3회 기준	7,091 3회 +3.5% = 차이 없음	7,880 3회 +15.1% ! 악화	7,767 3회 +13.4% ! 악화
유휴 RSS (healthz 3초 뒤) `bench.idleRssMB` · MiB · 값	354.5 3회 −1.0% = 차이 없음	358.2 3회 기준	361.2 3회 +0.8% = 차이 없음	377.8 3회 +5.5% = 차이 없음	371.9 3회 +3.8% = 차이 없음

v0 이전 (10-14, #1 머지) v1 현재 (4b 배포 기반) v1p 현재 + 개선 v2 다음 버전 후보 (v1.1) v2p 다음 버전 + 개선

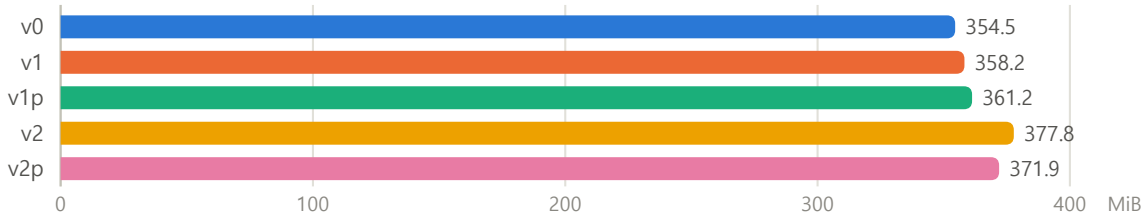
기동 (Popen→healthz 200) · bench.startupMs



Spring 'Started' 로그 시간 · bench.springStartedMs



유휴 RSS (healthz 3초 뒤) · bench.idleRssMB

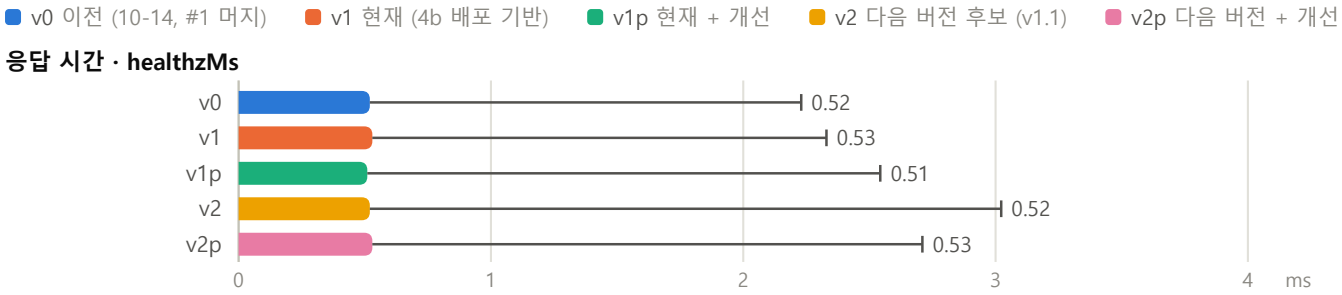


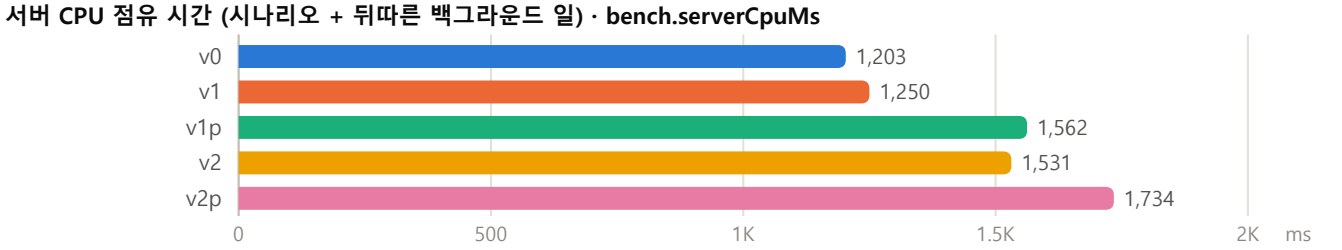
막대 = p50(반복을 합친 표본; 값 지표는 반복의 중앙값), 가는 선 = p95, 숫자 = p50. 지표마다 축이 따로다.

2. 헬스 체크 `healthz`

GET /api/v1/healthz 순차, keep-alive 연결 하나. 워밍업 100 + 측정 500.

지표	v0	v1	v1p	v2	v2p
응답 시간 `healthzMs` · ms · p50 / p95	0.52 / 2.23 n=1500 −1.9% = 차이 없음	0.53 / 2.33 n=1500 기준	0.51 / 2.54 n=1500 −3.8% = 차이 없음	0.52 / 3.02 n=1500 −1.9% = 차이 없음	0.53 / 2.71 n=1500 ±0.0% = 차이 없음
서버 CPU 점유 시간 (시나리오 + 뒤따른 백그라운드 일) `bench.serverCpuMs` · ms · 값	1,203 3회 −3.8%	1,250 3회 기준	1,562 3회 +25.0%	1,531 3회 +22.5%	1,734 3회 +38.8%
끝난 뒤 서버가 쉴 때까지 더 기다린 시간 (확인 창 제외) `bench.settleAfterMs` · ms · 값	0.59 3회	0.58 3회	1,001 3회	1,000 3회	1,001 3회



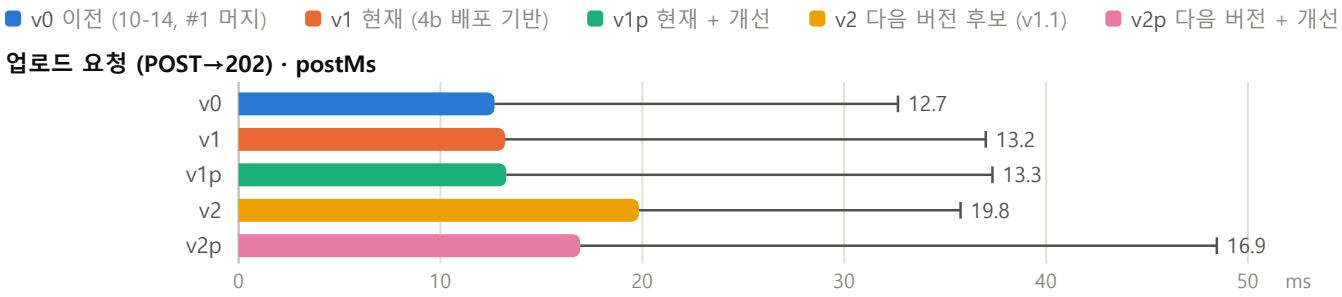


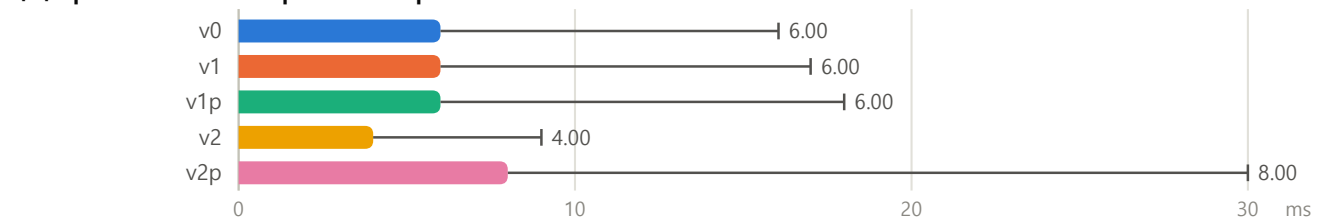
막대 = p50(반복을 합친 표본; 값 지표는 반복의 중앙값), 가는 선 = p95, 숫자 = p50. 지표마다 축이 따로다.

3. 작은 사진 업로드 `upload_small`

64×64 PNG(약 12 KB) 1장 POST /api/v1/jobs 순차. 워밍업 10 + 측정 100.

지표	v0	v1	v1p	v2	v2p
업로드 요청 (POST→202) `postMs` · ms · p50 / p95	12.7 / 32.7 n=300 −3.9% = 차이 없음	13.2 / 37.0 n=300 기준	13.3 / 37.3 n=300 +0.5% = 차이 없음	19.8 / 35.8 n=300 +50.3% ! 악화	16.9 / 48.5 n=300 +28.2% ! 악화
서버 uploadMs · events `postMs → uploadMs` · ms · p50 / p95	6.00 / 16.1 n=300 ±0.0% = 차이 없음	6.00 / 17.0 n=300 기준	6.00 / 18.0 n=300 ±0.0% = 차이 없음	4.00 / 9.00 n=300 −33.3% ✓ 개선	8.00 / 30.0 n=300 +33.3% ! 악화
서버 CPU 점유 시간 (시나리오 + 뒤따른 백그라운드 일) `bench.serverCpuMs` · ms · 값	4,219 3회 −2.5%	4,328 3회 기준	4,016 3회 −7.2%	5,047 3회 +16.6%	5,281 3회 +22.0%
끝난 뒤 서버가 쉴 때까지 더 기다린 시간 (확인 창 제외) `bench.settleAfterMs` · ms · 값	1,001 3회	0.46 3회	1,001 3회	0.44 3회	1,001 3회





서버 CPU 점유 시간 (시나리오 + 뒤따른 백그라운드 일) · bench.serverCpuMs



막대 = p50(반복을 합친 표본; 값 지표는 반복의 중앙값), 가는 선 = p95, 숫자 = p50. 지표마다 축이 따로다.

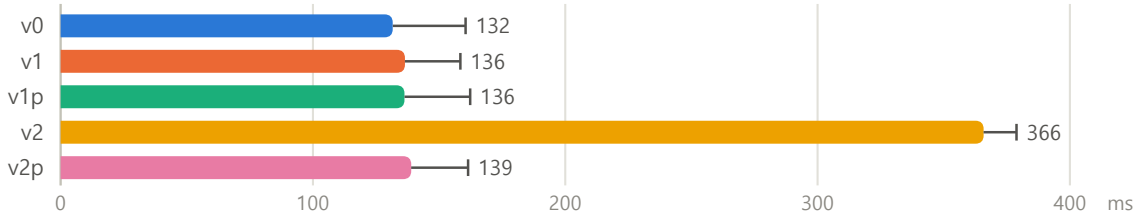
4. 큰 사진 업로드 `upload_large`

약 4.5 MB 노이즈 PNG 4장(요청 하나에 약 18 MB) POST 순차. 워밍업 2 + 측정 20, 요청당 4장.

지표	v0	v1	v1p	v2	v2p
업로드 요청 (POST→202) `postMs` · ms · p50 / p95	132 / 161 n=60 −3.5% = 차이 없음	136 / 158 n=60 기준	136 / 162 n=60 −0.1% = 차이 없음	366 / 379 n=60 +168.1% ! 악화	139 / 162 n=60 +1.8% = 차이 없음
업로드 처리량 (요청 바이트 ÷ p50) `bench.uploadMBps` · MB/s · 값	142.9 3회 +8.8% = 차이 없음	131.3 3회 기준	133.4 3회 +1.6% = 차이 없음	50.2 3회 −61.7% ! 악화	129.3 3회 −1.5% = 차이 없음
서버 uploadMs · events `postMs → uploadMs` · ms · p50 / p95	122 / 152 n=60 −3.2% = 차이 없음	126 / 148 n=60 기준	128 / 151 n=60 +1.2% = 차이 없음	130 / 146 n=60 +2.8% = 차이 없음	130 / 154 n=60 +3.2% = 차이 없음
서버 CPU 점유 시간 (시나리오 + 뒤따른 백그라운드 일) `bench.serverCpuMs` · ms · 값	3,641 3회 −12.1%	4,141 3회 기준	3,922 3회 −5.3%	4,594 3회 +10.9%	4,922 3회 +18.9%
끝난 뒤 서버가 쉴 때까지 더 기다린 시간 (확인 창 제외) `bench.settleAfterMs` · ms · 값	0.53 3회	0.64 3회	0.73 3회	0.33 3회	0.68 3회
측정 전마다 서버가 쉴 때까지 더 기다린 합계 (확인 창 제외) `bench.settleTotalMs` · ms · 값	2,511 3회	2,512 3회	2,511 3회	1,009 3회	8,015 3회

v0 이전 (10-14, #1 머지) v1 현재 (4b 배포 기반) v1p 현재 + 개선 v2 다음 버전 후보 (v1.1) v2p 다음 버전 + 개선

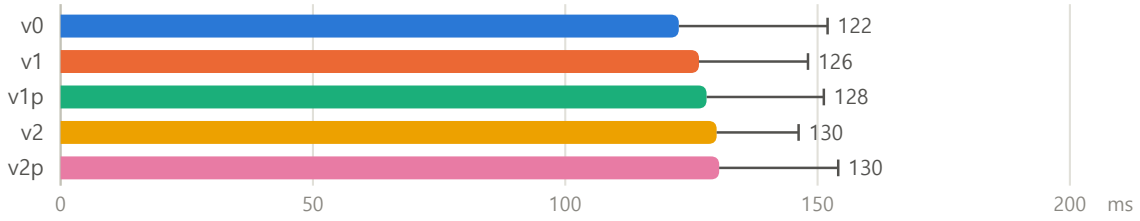
업로드 요청 (POST→202) · postMs



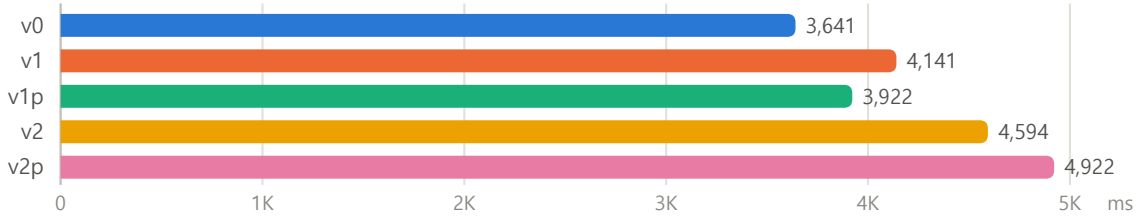
업로드 처리량 (요청 바이트 ÷ p50) · bench.uploadMBps



서버 uploadMs · events · postMs → uploadMs



서버 CPU 점유 시간 (시나리오 + 뒤따른 백그라운드 일) · bench.serverCpuMs



막대 = p50(반복을 합친 표본; 값 지표는 반복의 중앙값), 가는 선 = p95, 숫자 = p50. 지표마다 축이 따로다.

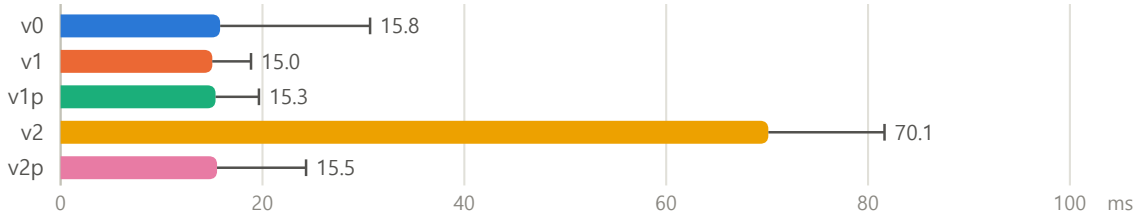
5. 12 MP 사진 업로드 `upload_12mp`

4000×3000 그라디언트 PNG(약 1.1 MB, 디코딩 부담) 1장 POST 순차, 이어서 4스레드 동시. 워밍업 2 + 측정 20, 동시 4스레드 × 5.

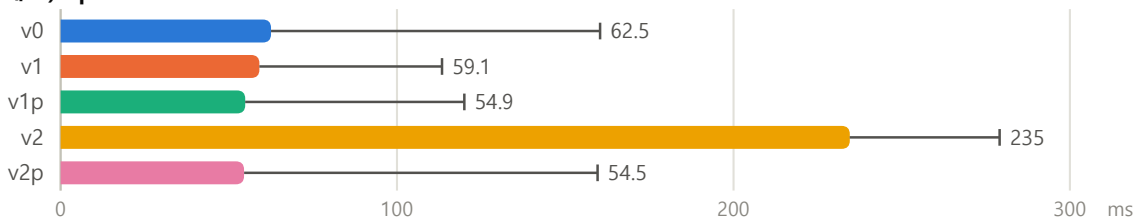
지표	v0	v1	v1p	v2	v2p
업로드 요청 (POST→202) `postMs` · ms · p50 / p95	15.8 / 30.7 n=60 +5.1% = 차이 없음	15.0 / 18.9 n=60 기준	15.3 / 19.7 n=60 +2.1% = 차이 없음	70.1 / 81.6 n=60 +366.5% ! 악화	15.5 / 24.3 n=60 +3.1% = 차이 없음
동시 업로드 (4스레드) `postConcurrentMs` · ms · p50 / p95	62.5 / 160 n=60 +5.9% = 차이 없음	59.1 / 113 n=60 기준	54.9 / 120 n=60 −7.1% = 차이 없음	235 / 279 n=60 +297.1% ! 악화	54.5 / 160 n=60 −7.7% = 차이 없음
서버 uploadMs · events `postMs → uploadMs` · ms · p50 / p95	11.0 / 19.2 n=60 ±0.0% = 차이 없음	11.0 / 13.0 n=60 기준	11.0 / 12.1 n=60 ±0.0% = 차이 없음	11.0 / 13.0 n=60 ±0.0% = 차이 없음	11.0 / 13.4 n=60 ±0.0% = 차이 없음
서버 uploadMs · events `postConcurrentMs → uploadMs` · ms · p50 / p95	42.5 / 103 n=60 +4.9% = 차이 없음	40.5 / 100 n=60 기준	45.0 / 100 n=60 +11.1% = 차이 없음	11.0 / 36.2 n=60 −72.8% ✓ 개선	38.5 / 122 n=60 −4.9% = 차이 없음
서버 CPU 점유 시간 (시나리오 + 뒤따른 백그라운드 일) `bench.serverCpuMs` · ms · 값	1,641 3회 −5.4%	1,734 3회 기준	1,875 3회 +8.1%	3,312 3회 +91.0%	3,672 3회 +111.7%
끝난 뒤 서버가 쉴 때까지 더 기다린 시간 (확인 창 제외) `bench.settleAfterMs` · ms · 값	0.64 3회	0.45 3회	1,001 3회	0.52 3회	1,001 3회
측정 전마다 서버가 쉴 때까지 더 기다린 합계 (확인 창 제외) `bench.settleTotalMs` · ms · 값	2,511 3회	4,511 3회	3,511 3회	1,511 3회	11,016 3회

v0 이전 (10-14, #1 머지) v1 현재 (4b 배포 기반) v1p 현재 + 개선 v2 다음 버전 후보 (v1.1) v2p 다음 버전 + 개선

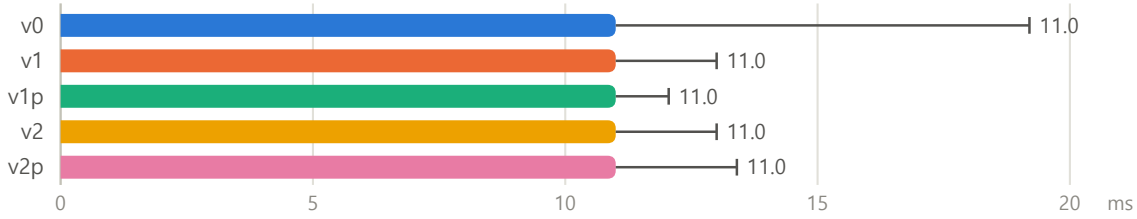
업로드 요청 (POST→202) · postMs



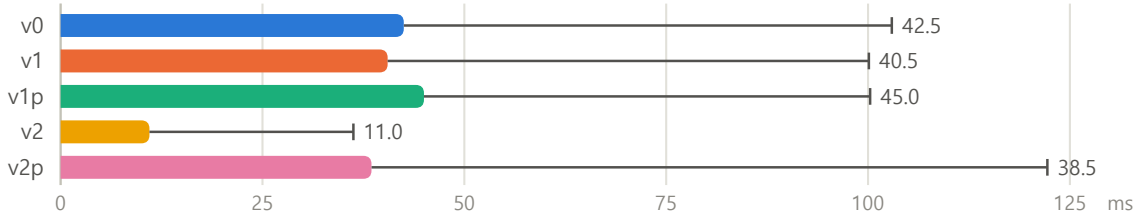
동시 업로드 (4스레드) · postConcurrentMs



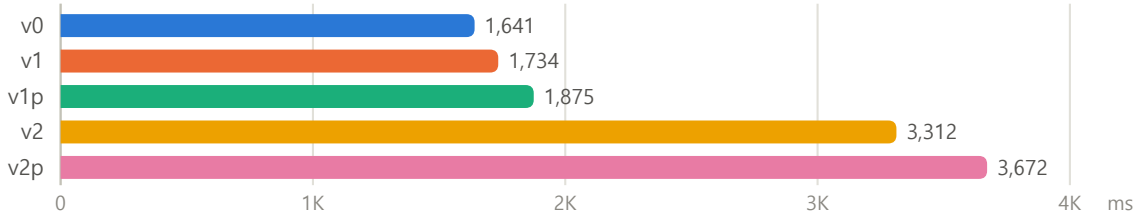
서버 uploadMs · events · postMs → uploadMs



서버 uploadMs · events · postConcurrentMs → uploadMs



서버 CPU 점유 시간 (시나리오 + 뒤따른 백그라운드 일) · bench.serverCpuMs

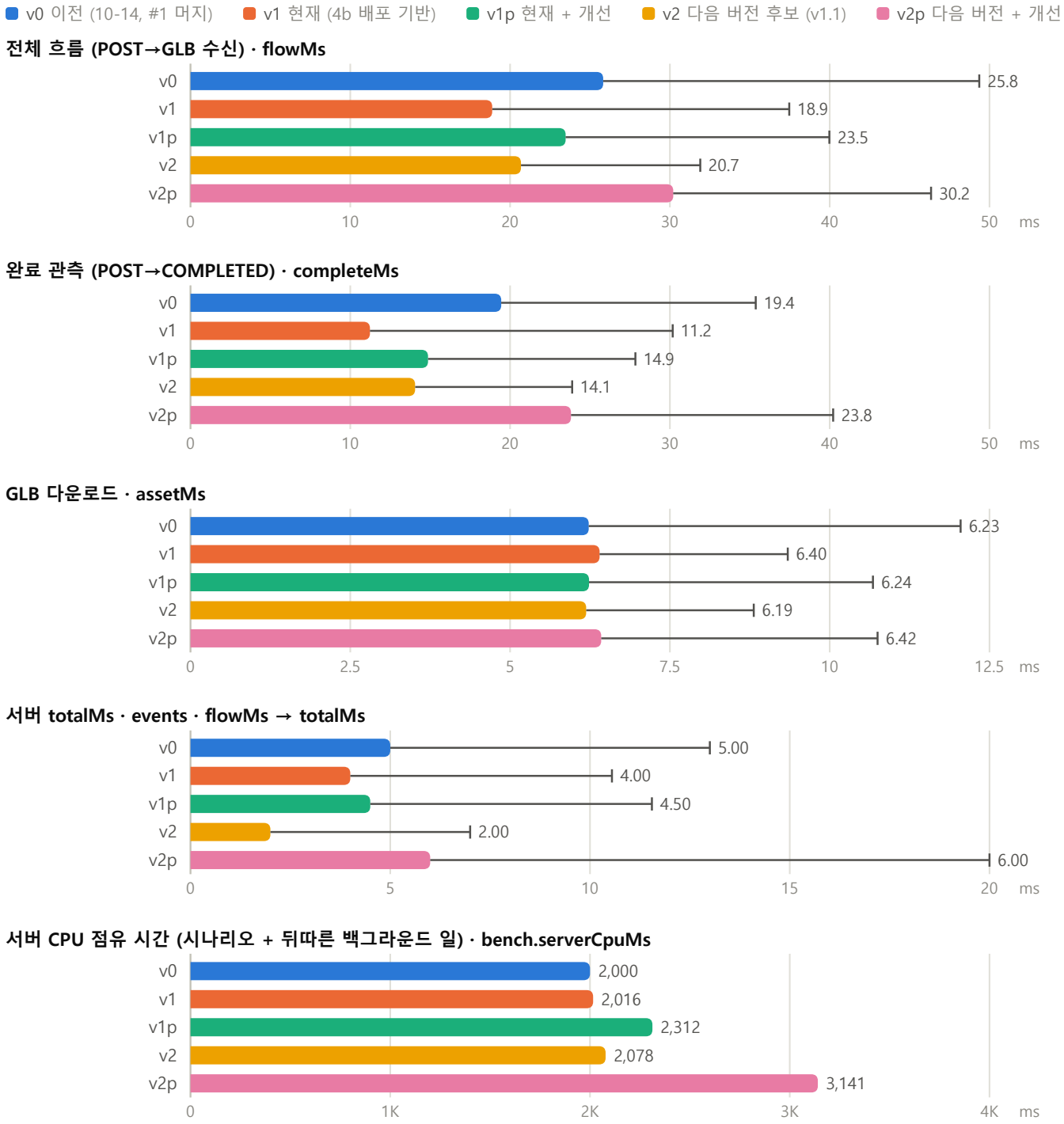


막대 = p50(반복을 합친 표본; 값 지표는 반복의 중앙값), 가는 선 = p95, 숫자 = p50. 지표마다 축이 따로다.

6. E2E 흐름 (mock) `e2e_mock`

POST → 10 ms 간격 폴링 → COMPLETED → GLB 다운로드(크기 확인), 순차. 워밍업 5 + 측정 50, 폴링 10 ms.

지표	v0	v1	v1p	v2	v2p
전체 흐름 (POST→GLB 수신) `flowMs` · ms · p50 / p95	25.8 / 49.4 n=150 +36.8% ! 악화	18.9 / 37.5 n=150 기준	23.5 / 40.0 n=150 +24.3% ! 악화	20.7 / 31.9 n=150 +9.5% = 차이 없음	30.2 / 46.3 n=150 +60.0% ! 악화
완료 관측 (POST→COMPLETED) `completeMs` · ms · p50 / p95	19.4 / 35.4 n=150 +73.2% ! 악화	11.2 / 30.2 n=150 기준	14.9 / 27.8 n=150 +32.4% = 차이 없음	14.1 / 23.9 n=150 +25.2% ! 악화	23.8 / 40.2 n=150 +112.0% ! 악화
GLB 다운로드 `assetMs` · ms · p50 / p95	6.23 / 12.0 n=150 −2.7% = 차이 없음	6.40 / 9.34 n=150 기준	6.24 / 10.7 n=150 −2.6% = 차이 없음	6.19 / 8.81 n=150 −3.3% = 차이 없음	6.42 / 10.7 n=150 +0.4% = 차이 없음
작업당 폴링 횟수 (평균) `bench.pollsPerJob` · 회 · 값	1.52 3회 +10.1%	1.38 3회 기준	1.46 3회 +5.8%	1.00 3회 −27.5%	1.62 3회 +17.4%
서버 totalMs · events `flowMs → totalMs` · ms · p50 / p95	5.00 / 13.0 n=150 +25.0% ! 악화	4.00 / 10.6 n=150 기준	4.50 / 11.6 n=150 +12.5% = 차이 없음	2.00 / 7.00 n=150 −50.0% = 차이 없음	6.00 / 20.0 n=150 +50.0% ! 악화
서버 CPU 점유 시간 (시나리오 + 뒤따른 백그라운드 일) `bench.serverCpuMs` · ms · 값	2,000 3회 −0.8%	2,016 3회 기준	2,312 3회 +14.7%	2,078 3회 +3.1%	3,141 3회 +55.8%
끝난 뒤 서버가 쉴 때까지 더 기다린 시간 (확인 창 제외) `bench.settleAfterMs` · ms · 값	0.57 3회	0.37 3회	0.36 3회	0.60 3회	0.67 3회



막대 = p50(반복을 합친 표본; 값 지표는 반복의 중앙값), 가는 선 = p95, 숫자 = p50. 지표마다 축이 따로다.

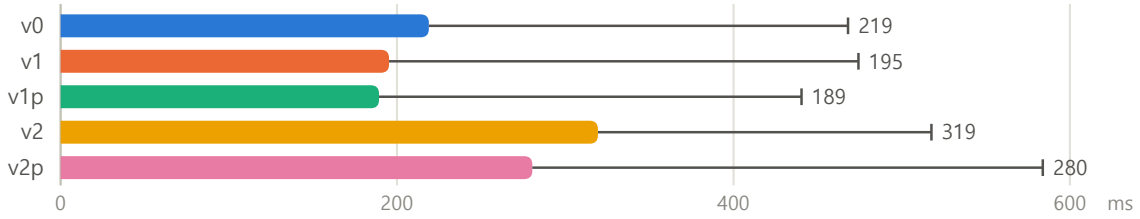
7. 동시 E2E 흐름 `concurrent_e2e`

16스레드가 각자 E2E 흐름을 반복, 폴링 간격 50 ms. 16스레드 × 10흐름, 폴링 50 ms.

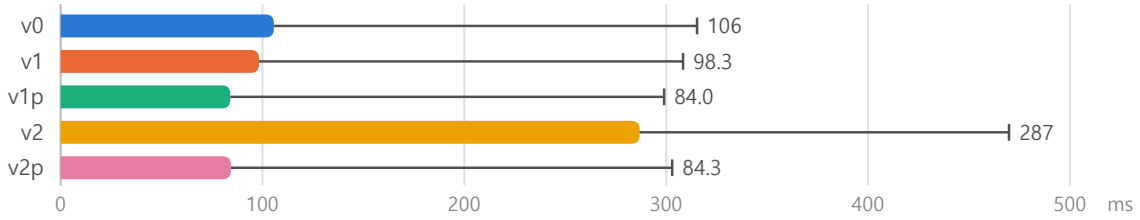
지표	v0	v1	v1p	v2	v2p
전체 흐름 (POST→GLB 수신) `flowMs` · ms · p50 / p95	219 / 468 n=480 +12.1% = 차이 없음	195 / 474 n=480 기준	189 / 440 n=480 −3.1% = 차이 없음	319 / 518 n=480 +63.6% ! 악화	280 / 584 n=480 +43.6% ! 악화
업로드 요청 (POST→202) `postMs` · ms · p50 / p95	106 / 315 n=480 +7.5% = 차이 없음	98.3 / 308 n=480 기준	84.0 / 299 n=480 −14.5% = 차이 없음	287 / 470 n=480 +191.9% ! 악화	84.3 / 303 n=480 −14.2% = 차이 없음
상태 조회 GET /jobs/{id} `pollMs` · ms · p50 / p95	5.17 / 58.0 n=1009 +2.4% = 차이 없음	5.05 / 58.3 n=966 기준	4.54 / 61.1 n=951 −10.1% = 차이 없음	2.35 / 92.9 n=568 −53.5% ✓ 개선	18.1 / 136 n=1153 +258.0% ! 악화
GLB 다운로드 `assetMs` · ms · p50 / p95	14.8 / 86.7 n=480 −3.7% = 차이 없음	15.4 / 90.2 n=480 기준	16.9 / 86.9 n=480 +9.9% = 차이 없음	10.2 / 58.8 n=480 −34.0% ✓ 개선	18.4 / 106 n=480 +19.2% = 차이 없음
처리량 (완료 흐름 ÷ 경과 시간) `bench.jobsPerSec` · 건/s · 값	64.9 3회 +4.4% = 차이 없음	62.2 3회 기준	67.0 3회 +7.8% = 차이 없음	47.2 3회 −24.0% ! 악화	47.1 3회 −24.3% ! 악화
오류 (실패한 요청·흐름) `bench.errors` · 건 · 값	0 3회 합계	0 3회 합계	0 3회 합계	0 3회 합계	0 3회 합계
서버 totalMs · events `flowMs → totalMs` · ms · p50 / p95	68.0 / 275 n=480 +17.2% ! 악화	58.0 / 250 n=480 기준	55.0 / 262 n=480 −5.2% = 차이 없음	35.0 / 202 n=480 −39.7% ✓ 개선	111 / 337 n=480 +91.4% ! 악화
서버 CPU 점유 시간 (시나리오 + 뒤따른 백그라운드 일) `bench.serverCpuMs` · ms · 값	5,000 3회 +1.3%	4,938 3회 기준	4,750 3회 −3.8%	6,750 3회 +36.7%	7,031 3회 +42.4%
끝난 뒤 서버가 쉴 때까지 더 기다린 시간 (확인 창 제외) `bench.settleAfterMs` · ms · 값	0.57 3회	0.47 3회	0.41 3회	0.56 3회	1,001 3회

v0 이전 (10-14, #1 머지) · v1 현재 (4b 배포 기반) · v1p 현재 + 개선 · v2 다음 버전 후보 (v1.1) · v2p 다음 버전 + 개선

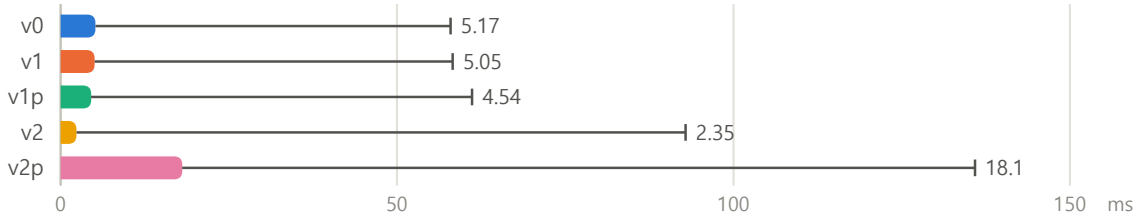
전체 흐름 (POST→GLB 수신) · flowMs



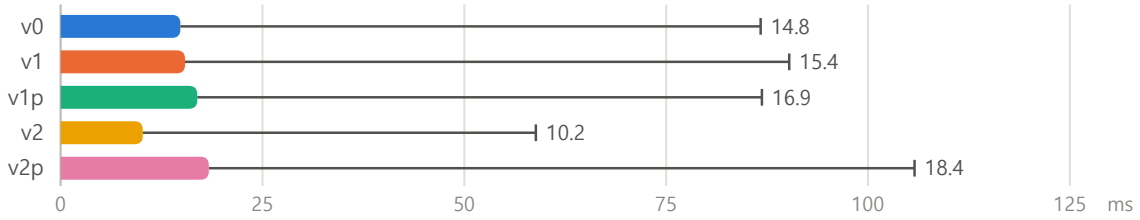
업로드 요청 (POST→202) · postMs



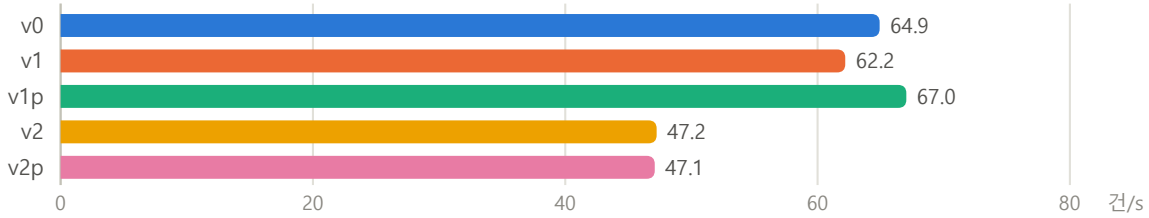
상태 조회 GET /jobs/{id} · pollMs



GLB 다운로드 · assetMs



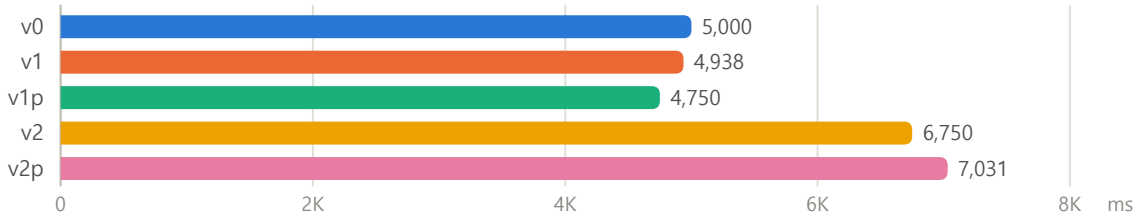
처리량 (완료 흐름 ÷ 경과 시간) · bench.jobsPerSec



서버 totalMs · events · flowMs → totalMs



서버 CPU 점유 시간 (시나리오 + 뒤따른 백그라운드 일) · bench.serverCpuMs



막대 = p50(반복을 합친 표본; 값 지표는 반복의 중앙값), 가는 선 = p95, 숫자 = p50. 지표마다 축이 따로다.

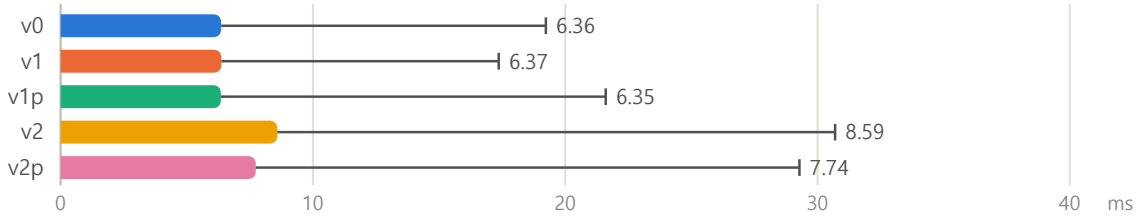
8. 폴링 부하 `poll_load`

COMPLETED 작업 하나를 4프로세스 × 4스레드가 GET /api/v1/jobs/{id} 로 반복 조회. 4프로세스 × 4스레드 × 250 (스레드마다 연결용 워밍업 5).

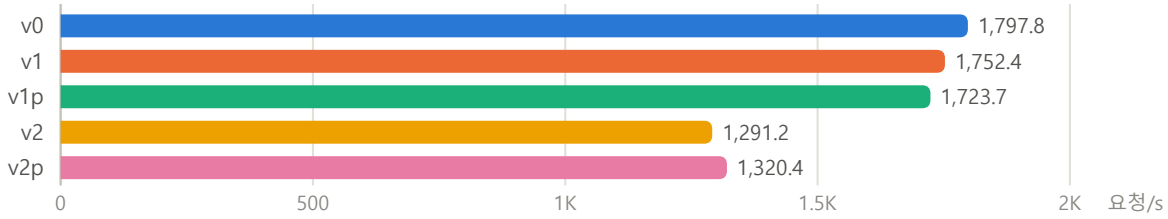
지표	v0	v1	v1p	v2	v2p
상태 조회 GET /jobs/{id} `pollMs` · ms · p50 / p95	6.36 / 19.2 n=12000 −0.2% = 차이 없음	6.37 / 17.4 n=12000 기준	6.35 / 21.6 n=12000 −0.3% = 차이 없음	8.59 / 30.7 n=12000 +34.9% ! 악화	7.74 / 29.3 n=12000 +21.4% = 차이 없음
처리량 (성공 요청 ÷ 경과 시간) `bench.pollReqPerSec` · 요청/s · 값	1,797.8 3회 +2.6% = 차이 없음	1,752.4 3회 기준	1,723.7 3회 −1.6% = 차이 없음	1,291.2 3회 −26.3% ! 악화	1,320.4 3회 −24.6% ! 악화
오류 (실패한 요청·흐름) `bench.errors` · 건 · 값	0 3회 합계	0 3회 합계	0 3회 합계	0 3회 합계	0 3회 합계
서버 CPU 점유 시간 (시나리오 + 뒤따른 백그라운드 일) `bench.serverCpuMs` · ms · 값	4,531 3회 −4.9%	4,766 3회 기준	4,812 3회 +1.0%	6,375 3회 +33.8%	6,188 3회 +29.8%
끝난 뒤 서버가 쉴 때까지 더 기다린 시간 (확인 창 제외) `bench.settleAfterMs` · ms · 값	0.45 3회	0.48 3회	1,001 3회	0.23 3회	1,001 3회

v0 이전 (10-14, #1 머지) v1 현재 (4b 배포 기반) v1p 현재 + 개선 v2 다음 버전 후보 (v1.1) v2p 다음 버전 + 개선

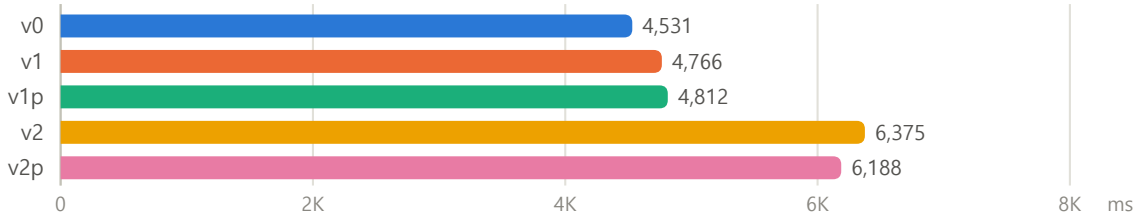
상태 조회 GET /jobs/{id} · pollMs



처리량 (성공 요청 ÷ 경과 시간) · bench.pollReqPerSec



서버 CPU 점유 시간 (시나리오 + 뒤따른 백그라운드 일) · bench.serverCpuMs

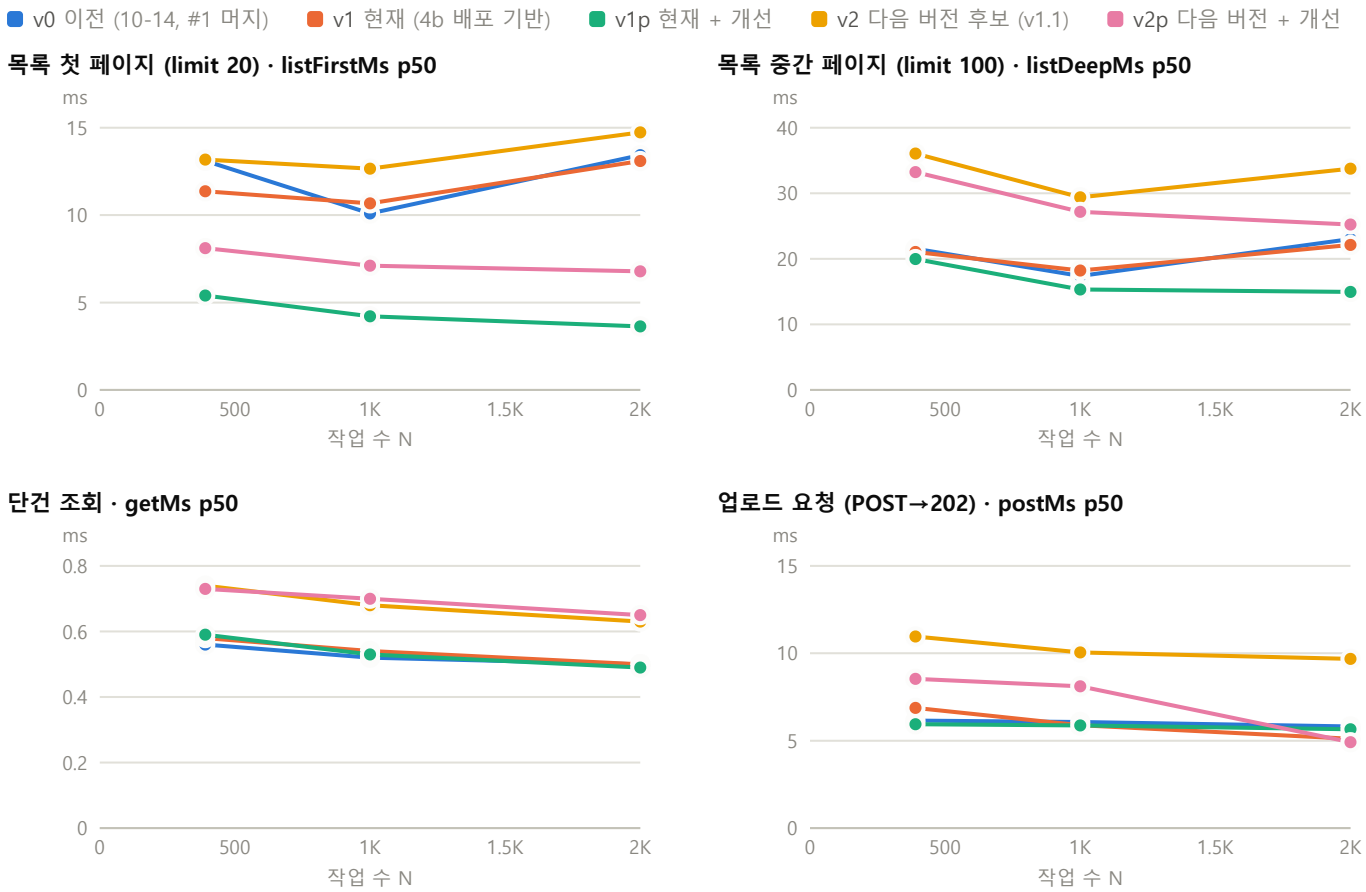


막대 = p50(반복을 합친 표본; 값 지표는 반복의 중앙값), 가는 선 = p95, 숫자 = p50. 지표마다 축이 따로다.

9. 작업 수에 따른 변화 `scale`

작업을 N 개까지 쌓은 뒤(8스레드 업로드) 목록 첫 페이지·중간 페이지, 단건 조회, 업로드. N = 200, 1000, 2000; 목록 첫 100, 중간 50, 조회 200, 업로드 30 (각 워밍업 5, 업로드 제외).

지표	v0	v1	v1p	v2	v2p
목록 첫 페이지 (limit 20) · N=390 (목표 200) `listFirstMs@200` · ms · p50 / p95	13.1 / 18.0 n=300 +15.3% = 차이 없음	11.4 / 16.5 n=300 기준	5.40 / 8.30 n=300 −52.5% ✓ 개선	13.2 / 17.8 n=300 +15.9% ! 악화	8.12 / 10.9 n=300 −28.6% ✓ 개선
목록 첫 페이지 (limit 20) · N=1000 `listFirstMs@1000` · ms · p50 / p95	10.1 / 13.5 n=300 −5.4% = 차이 없음	10.7 / 13.6 n=300 기준	4.21 / 7.46 n=300 −60.5% ✓ 개선	12.7 / 16.1 n=300 +18.6% ! 악화	7.11 / 8.63 n=300 −33.4% ✓ 개선
목록 첫 페이지 (limit 20) · N=2000 `listFirstMs@2000` · ms · p50 / p95	13.4 / 20.7 n=300 +2.6% = 차이 없음	13.1 / 20.0 n=300 기준	3.64 / 4.65 n=300 −72.2% ✓ 개선	14.7 / 21.3 n=300 +12.6% = 차이 없음	6.78 / 8.54 n=300 −48.2% ✓ 개선
목록 중간 페이지 (limit 100) · N=390 (목표 200) `listDeepMs@200` · ms · p50 / p95	21.5 / 29.7 n=150 +2.4% = 차이 없음	21.0 / 27.2 n=150 기준	20.0 / 27.5 n=150 −5.1% = 차이 없음	36.0 / 40.8 n=150 +71.4% ! 악화	33.2 / 39.5 n=150 +58.0% ! 악화
목록 중간 페이지 (limit 100) · N=1000 `listDeepMs@1000` · ms · p50 / p95	17.4 / 23.9 n=150 −4.5% = 차이 없음	18.2 / 24.2 n=150 기준	15.3 / 22.9 n=150 −16.0% ✓ 개선	29.4 / 39.5 n=150 +61.2% ! 악화	27.2 / 34.8 n=150 +49.0% ! 악화
목록 중간 페이지 (limit 100) · N=2000 `listDeepMs@2000` · ms · p50 / p95	23.0 / 31.9 n=150 +4.1% = 차이 없음	22.1 / 29.8 n=150 기준	15.0 / 18.2 n=150 −32.3% ✓ 개선	33.8 / 39.5 n=150 +52.7% ! 악화	25.2 / 31.1 n=150 +14.1% ! 악화
단건 조회 · N=390 (목표 200) `getMs@200` · ms · p50 / p95	0.56 / 0.86 n=600 −3.5% = 차이 없음	0.58 / 0.86 n=600 기준	0.59 / 0.87 n=600 +1.7% = 차이 없음	0.74 / 1.03 n=600 +27.6% ! 악화	0.73 / 0.97 n=600 +25.9% ! 악화
단건 조회 · N=1000 `getMs@1000` · ms · p50 / p95	0.52 / 0.67 n=600 −3.7% = 차이 없음	0.54 / 0.76 n=600 기준	0.53 / 0.66 n=600 −1.9% = 차이 없음	0.68 / 0.95 n=600 +25.9% ! 악화	0.70 / 0.97 n=600 +29.6% ! 악화
단건 조회 · N=2000 `getMs@2000` · ms · p50 / p95	0.50 / 0.88 n=600 ±0.0% = 차이 없음	0.50 / 0.66 n=600 기준	0.49 / 0.78 n=600 −2.0% = 차이 없음	0.63 / 0.79 n=600 +26.0% ! 악화	0.65 / 0.85 n=600 +30.0% ! 악화
업로드 요청 (POST→202) · N=390 (목표 200) `postMs@200` · ms · p50 / p95	6.15 / 13.5 n=90 −10.6% = 차이 없음	6.88 / 12.8 n=90 기준	5.95 / 13.8 n=90 −13.6% ✓ 개선	11.0 / 14.1 n=90 +59.3% ! 악화	8.54 / 16.5 n=90 +24.1% ! 악화
업로드 요청 (POST→202) · N=1000 `postMs@1000` · ms · p50 / p95	6.07 / 8.64 n=90 +3.1% = 차이 없음	5.88 / 9.81 n=90 기준	5.87 / 8.95 n=90 −0.2% = 차이 없음	10.1 / 12.2 n=90 +70.9% ! 악화	8.12 / 22.2 n=90 +37.9% ! 악화
업로드 요청 (POST→202) · N=2000 `postMs@2000` · ms · p50 / p95	5.82 / 8.55 n=90 +13.8% = 차이 없음	5.11 / 8.02 n=90 기준	5.66 / 7.64 n=90 +10.7% = 차이 없음	9.68 / 11.3 n=90 +89.3% ! 악화	4.92 / 8.71 n=90 −3.7% = 차이 없음
실제 작업 수 (목표 N=200) `bench.actualN@200` · 개 · 값	390 3회	390 3회	390 3회	390 3회	390 3회
실제 작업 수 (목표 N=1000) `bench.actualN@1000` · 개 · 값	1,000 3회	1,000 3회	1,000 3회	1,000 3회	1,000 3회
실제 작업 수 (목표 N=2000) `bench.actualN@2000` · 개 · 값	2,000 3회	2,000 3회	2,000 3회	2,000 3회	2,000 3회

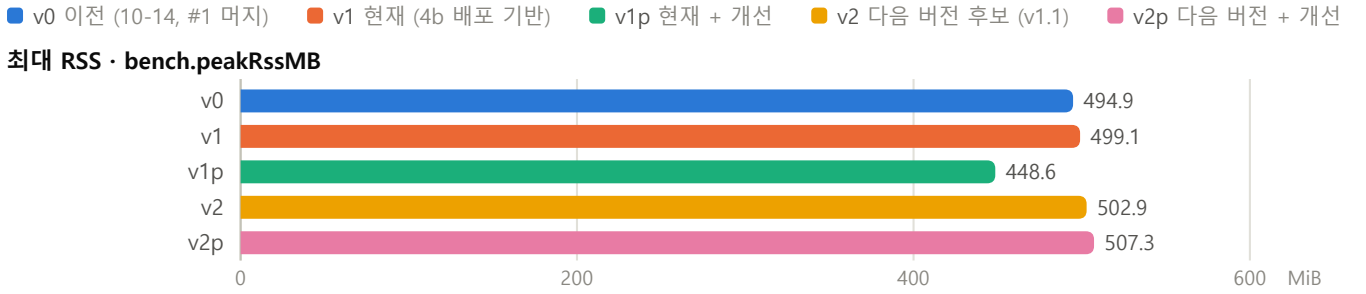


가로축 = 측정 때 서버에 있던 작업 수(이전 시나리오가 만든 작업 포함), 세로축 = p50. 목록은 최신 작업부터, 중간 페이지는 서버가 준 nextCursor 로 약 N/2 위치에서 시작.

10. 자원 `resources`

mock 시나리오를 모두 마친 서버 A 의 최대 RSS(작업 집합 최대치)와 jar 크기. 1회.

지표	v0	v1	v1p	v2	v2p
최대 RSS `bench.peakRssMB` · MiB · 값	494.9 3회 −0.8% = 차이 없음	499.1 3회 기준	448.6 3회 −10.1% ✓ 개선	502.9 3회 +0.8% = 차이 없음	507.3 3회 +1.7% = 차이 없음
jar 크기 `bench.jarBytes` · 값	52.9 MiB 3회 −0.1%	53.0 MiB 3회 기준	53.0 MiB 3회 ±0.0%	53.8 MiB 3회 +1.5%	53.8 MiB 3회 +1.5%



막대 = p50(반복을 합친 표본; 값 지표는 반복의 중앙값), 가는 선 = p95, 숫자 = p50. 지표마다 축이 따로다.

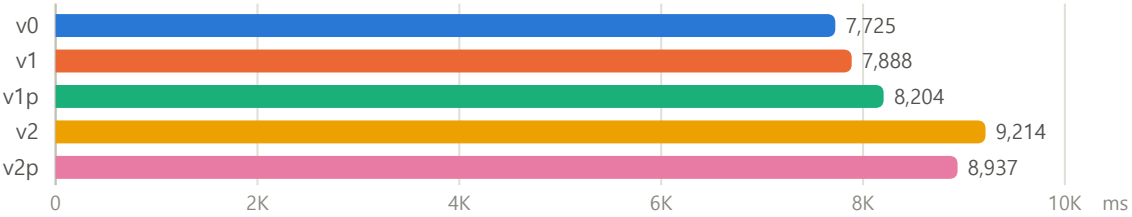
11. 재기동·PENDING 폴링 `pending_poll`

서버 A 의 저장소로 재기동(서버 B, 작업이 PENDING 에 머묾): 기동 시간, PENDING 작업 조회. PENDING 작업 5개, 워밍업 20 + 측정 200.

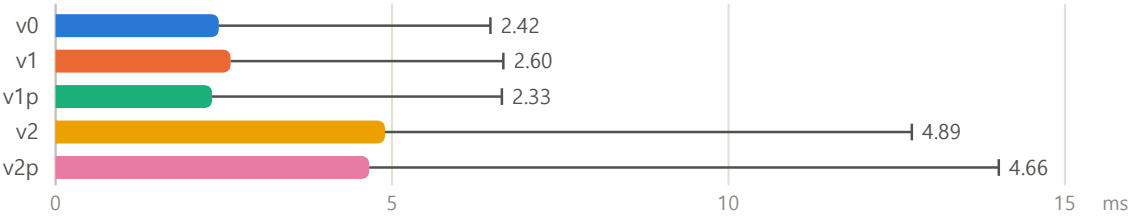
지표	v0	v1	v1p	v2	v2p
재기동 (Popen→healthz 200) `bench.restartStartupMs` · ms · 값	7,725 3회 −2.1% = 차이 없음	7,888 3회 기준	8,204 3회 +4.0% = 차이 없음	9,214 3회 +16.8% ! 악화	8,937 3회 +13.3% ! 악화
PENDING 작업 조회 `pendingPollMs` · ms · p50 / p95	2.42 / 6.46 n=600 −6.7% = 차이 없음	2.60 / 6.65 n=600 기준	2.33 / 6.63 n=600 −10.6% = 차이 없음	4.89 / 12.7 n=600 +88.3% ! 악화	4.66 / 14.0 n=600 +79.2% ! 악화

v0 이전 (10-14, #1 머지) · v1 현재 (4b 배포 기반) · v1p 현재 + 개선 · v2 다음 버전 후보 (v1.1) · v2p 다음 버전 + 개선

재기동 (Popen→healthz 200) · bench.restartStartupMs



PENDING 작업 조회 · pendingPollMs



막대 = p50(반복을 합친 표본; 값 지표는 반복의 중앙값), 가는 선 = p95, 숫자 = p50. 지표마다 축이 따로다.

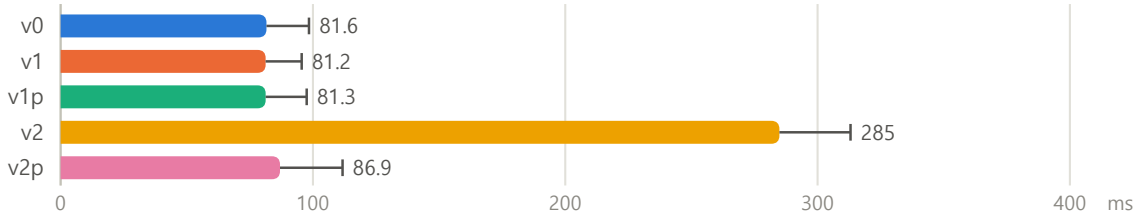
12. real · path 전송 `real_path`

real 프로파일 + 추론 스텁(패스스루, 모델 없음), 사진 경로 전달. 4.5 MB PNG 1장 E2E 순차, 이어서 5건 동시 접수. 워밍업 3 + 측정 20, 폴링 20 ms, 동시 접수 5건.

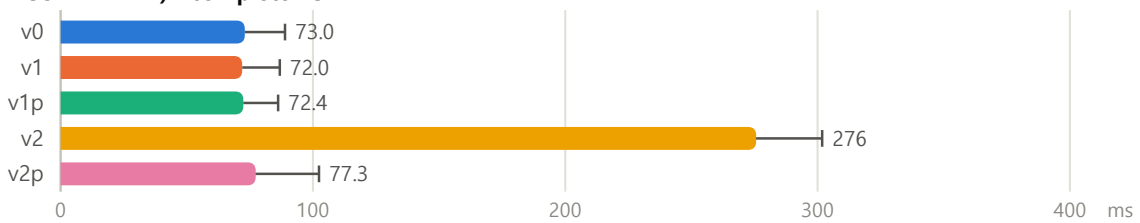
지표	v0	v1	v1p	v2	v2p
전체 흐름 (POST→GLB 수신) `flowMs` · ms · p50 / p95	81.6 / 98.4 n=60 +0.4% = 차이 없음	81.2 / 95.5 n=60 기준	81.3 / 97.5 n=60 ±0.0% = 차이 없음	285 / 313 n=60 +250.6% ! 악화	86.9 / 112 n=60 +7.0% = 차이 없음
완료 관측 (POST→COMPLETED) `completeMs` · ms · p50 / p95	73.0 / 88.9 n=60 +1.4% = 차이 없음	72.0 / 86.9 n=60 기준	72.4 / 86.2 n=60 +0.6% = 차이 없음	276 / 302 n=60 +283.1% ! 악화	77.3 / 102 n=60 +7.4% = 차이 없음
GLB 다운로드 `assetMs` · ms · p50 / p95	8.74 / 11.3 n=60 +1.2% = 차이 없음	8.63 / 11.3 n=60 기준	8.82 / 12.8 n=60 +2.1% = 차이 없음	9.11 / 12.2 n=60 +5.4% = 차이 없음	8.86 / 12.7 n=60 +2.6% = 차이 없음
업로드 요청 (POST→202) `postMs` · ms · p50 / p95	44.1 / 56.3 n=60 +2.4% = 차이 없음	43.1 / 55.9 n=60 기준	41.9 / 54.8 n=60 −2.8% = 차이 없음	272 / 297 n=60 +531.6% ! 악화	44.7 / 55.3 n=60 +3.9% = 차이 없음
서버 queuedMs `serverQueuedMs` · ms · p50 / p95	13.0 / 22.1 n=60 ±0.0% = 차이 없음	13.0 / 22.1 n=60 기준	12.5 / 26.1 n=60 −3.9% = 차이 없음	12.0 / 18.1 n=60 −7.7% = 차이 없음	13.0 / 27.0 n=60 ±0.0% = 차이 없음
서버 processingMs `serverProcessingMs` · ms · p50 / p95	12.5 / 23.0 n=60 +4.2% = 차이 없음	12.0 / 22.1 n=60 기준	11.0 / 20.0 n=60 −8.3% = 차이 없음	10.0 / 17.1 n=60 −16.7% = 차이 없음	15.0 / 27.2 n=60 +25.0% = 차이 없음
서버 totalMs `serverTotalMs` · ms · p50 / p95	26.0 / 40.0 n=60 ±0.0% = 차이 없음	26.0 / 38.1 n=60 기준	26.0 / 42.0 n=60 ±0.0% = 차이 없음	22.0 / 37.0 n=60 −15.4% = 차이 없음	28.5 / 48.2 n=60 +9.6% = 차이 없음
작업 사이 빈 시간 `queueGapMs` · ms · p50 / p95	5.50 / 11.1 n=12 −29.1% = 차이 없음	7.76 / 21.3 n=12 기준	10.3 / 16.6 n=12 +33.4% ! 악화	6.15 / 8.37 n=12 −20.8% = 차이 없음	10.6 / 23.9 n=12 +36.3% ! 악화
기동 (Popen→healthz 200) `bench.startupMs` · ms · 값	7,983 3회 −3.2% = 차이 없음	8,250 3회 기준	8,066 3회 −2.2% = 차이 없음	9,025 3회 +9.4% = 차이 없음	8,930 3회 +8.2% = 차이 없음
서버 CPU 점유 시간 (시나리오 + 뒤따른 백그라운드 일) `bench.serverCpuMs` · ms · 값	4,391 3회 +8.1%	4,062 3회 기준	4,688 3회 +15.4%	6,359 3회 +56.5%	6,188 3회 +52.3%
측정 전마다 서버가 쉴 때까지 더 기다린 합계 (확인 창 제외) `bench.settleTotalMs` · ms · 값	1,511 3회	1,511 3회	4,511 3회	2,512 3회	7,512 3회

v0 이전 (10-14, #1 머지) v1 현재 (4b 배포 기반) v1p 현재 + 개선 v2 다음 버전 후보 (v1.1) v2p 다음 버전 + 개선

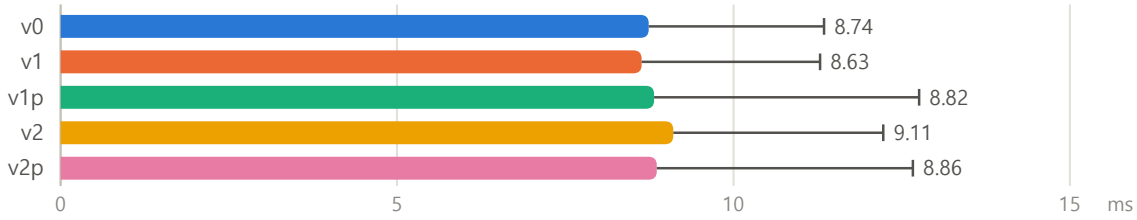
전체 흐름 (POST→GLB 수신) · flowMs



완료 관측 (POST→COMPLETED) · completeMs



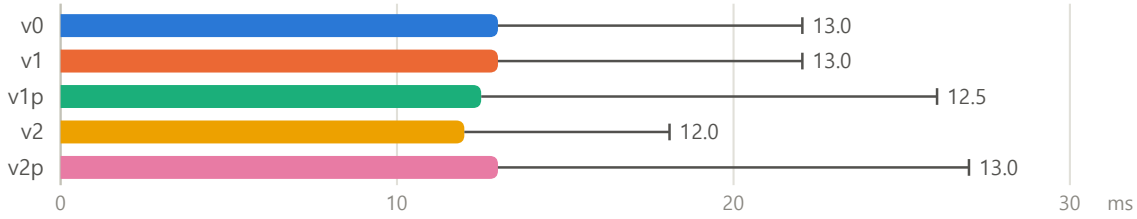
GLB 다운로드 · assetMs



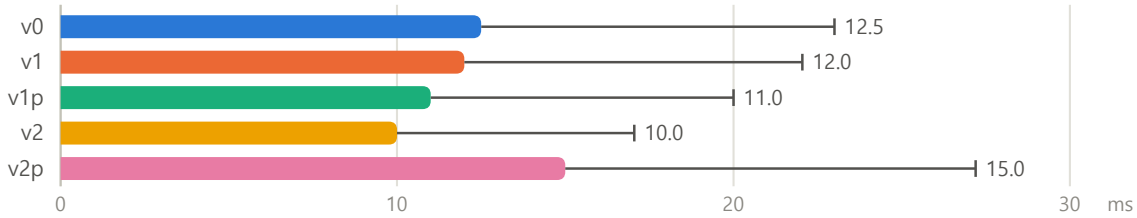
업로드 요청 (POST→202) · postMs



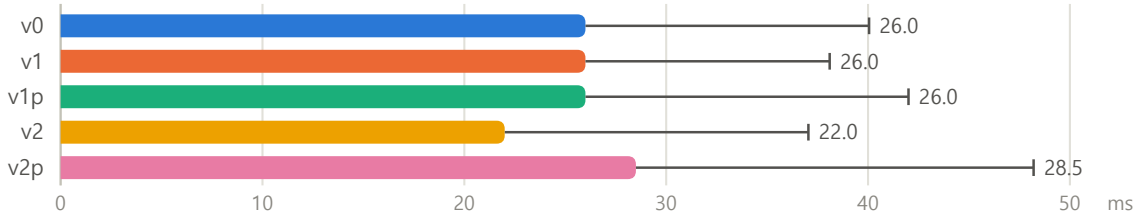
서버 queuedMs · serverQueuedMs



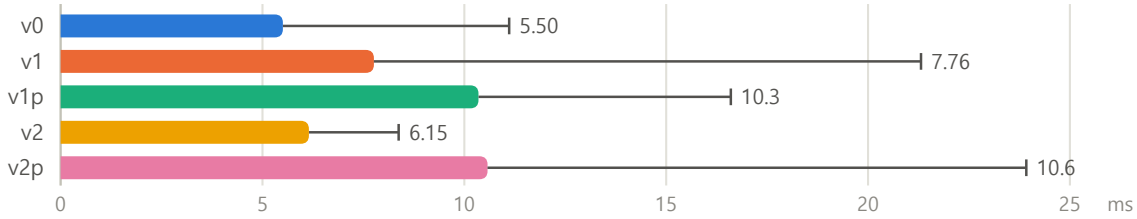
서버 processingMs · serverProcessingMs



서버 totalMs · serverTotalMs



작업 사이 빈 시간 · queueGapMs



막대 = p50(반복을 합친 표본; 값 지표는 반복의 중앙값), 가는 선 = p95, 숫자 = p50. 지표마다 축이 따로다.

13. real · upload 전송 `real_upload`

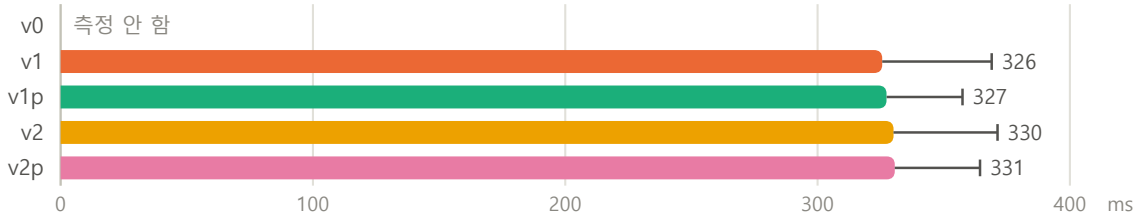
real_path 와 같고 /infer/upload 로 사진을 올리고 GLB 를 내려받는다. 워밍업 3 + 측정 20, 폴링 20 ms, 동시 접수 5건.

v0: 측정 안 함 — 이 버전에 없는 기능 (real_upload)

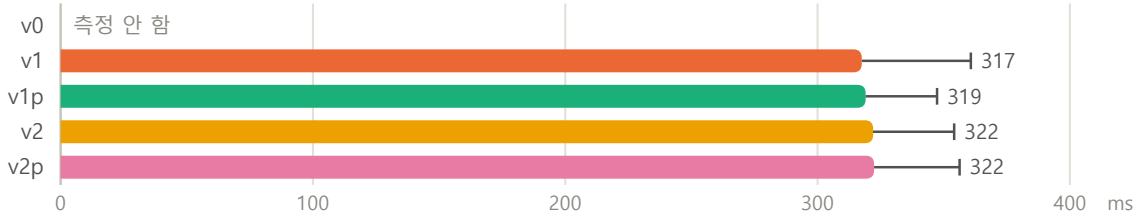
지표	v0	v1	v1p	v2	v2p
전체 흐름 (POST→GLB 수신) `flowMs` · ms · p50 / p95	측정 안 함 이 버전에 없는 기능 (real_upload)	326 / 369 n=60 기준	327 / 357 n=60 +0.5% = 차이 없음	330 / 371 n=60 +1.4% = 차이 없음	331 / 364 n=60 +1.5% = 차이 없음
완료 관측 (POST→COMPLETED) `completeMs` · ms · p50 / p95	측정 안 함 이 버전에 없는 기능 (real_upload)	317 / 361 n=60 기준	319 / 347 n=60 +0.5% = 차이 없음	322 / 354 n=60 +1.4% = 차이 없음	322 / 356 n=60 +1.5% = 차이 없음
GLB 다운로드 `assetMs` · ms · p50 / p95	측정 안 함 이 버전에 없는 기능 (real_upload)	8.05 / 10.8 n=60 기준	8.20 / 10.2 n=60 +1.8% = 차이 없음	9.17 / 14.9 n=60 +13.9% ! 악화	8.20 / 11.2 n=60 +1.8% = 차이 없음
업로드 요청 (POST→202) `postMs` · ms · p50 / p95	측정 안 함 이 버전에 없는 기능 (real_upload)	42.6 / 54.6 n=60 기준	42.2 / 54.4 n=60 −0.9% = 차이 없음	278 / 311 n=60 +551.2% ! 악화	42.8 / 56.1 n=60 +0.4% = 차이 없음
서버 queuedMs `serverQueuedMs` · ms · p50 / p95	측정 안 함 이 버전에 없는 기능 (real_upload)	11.0 / 20.1 n=60 기준	14.0 / 23.0 n=60 +27.3% = 차이 없음	13.0 / 21.0 n=60 +18.2% = 차이 없음	13.0 / 27.1 n=60 +18.2% = 차이 없음
서버 processingMs `serverProcessingMs` · ms · p50 / p95	측정 안 함 이 버전에 없는 기능 (real_upload)	263 / 298 n=60 기준	262 / 287 n=60 −0.2% = 차이 없음	260 / 296 n=60 −1.1% = 차이 없음	263 / 303 n=60 ±0.0% = 차이 없음
서버 totalMs `serverTotalMs` · ms · p50 / p95	측정 안 함 이 버전에 없는 기능 (real_upload)	277 / 314 n=60 기준	276 / 298 n=60 −0.4% = 차이 없음	274 / 312 n=60 −0.9% = 차이 없음	279 / 324 n=60 +0.7% = 차이 없음
작업 사이 빈 시간 `queueGapMs` · ms · p50 / p95	측정 안 함 이 버전에 없는 기능 (real_upload)	12.5 / 21.7 n=12 기준	15.9 / 41.3 n=12 +27.1% = 차이 없음	12.1 / 16.9 n=12 −3.1% = 차이 없음	16.9 / 32.0 n=12 +34.9% ! 악화
기동 (Popen→healthz 200) `bench.startupMs` · ms · 값	측정 안 함 이 버전에 없는 기능 (real_upload)	8,110 3회 기준	9,044 3회 +11.5% = 차이 없음	9,036 3회 +11.4% ! 악화	9,195 3회 +13.4% ! 악화
서버 CPU 점유 시간 (시나리오 + 뒤따른 백그라운드 일) `bench.serverCpuMs` · ms · 값	측정 안 함 이 버전에 없는 기능 (real_upload)	7,578 3회 기준	8,000 3회 +5.6%	7,562 3회 −0.2%	9,672 3회 +27.6%
측정 전마다 서버가 쉴 때까지 더 기다린 합계 (확인 창 제외) `bench.settleTotalMs` · ms · 값	측정 안 함 이 버전에 없는 기능 (real_upload)	2,513 3회	2,511 3회	3,011 3회	3,012 3회

v0 이전 (10-14, #1 머지) · v1 현재 (4b 배포 기반) · v1p 현재 + 개선 · v2 다음 버전 후보 (v1.1) · v2p 다음 버전 + 개선

전체 흐름 (POST→GLB 수신) · flowMs



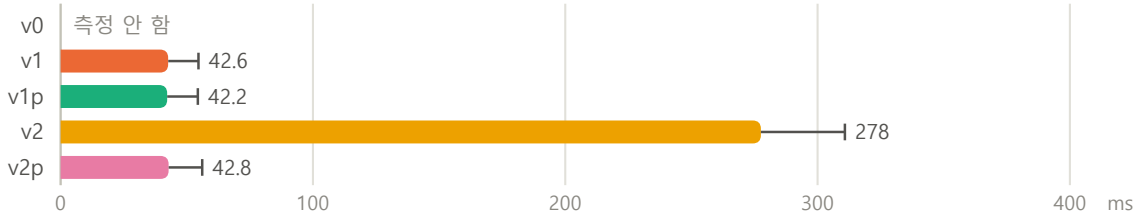
완료 관측 (POST→COMPLETED) · completeMs



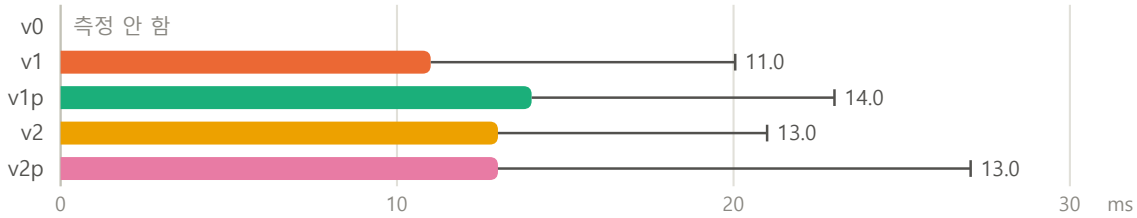
GLB 다운로드 · assetMs



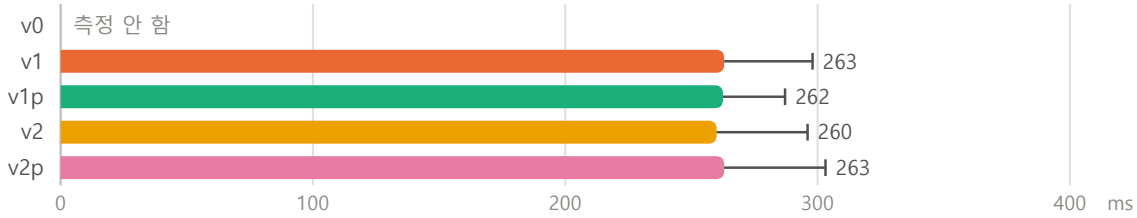
업로드 요청 (POST→202) · postMs



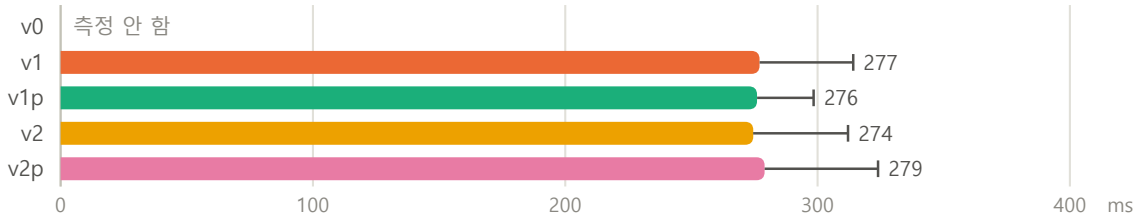
서버 queuedMs · serverQueuedMs



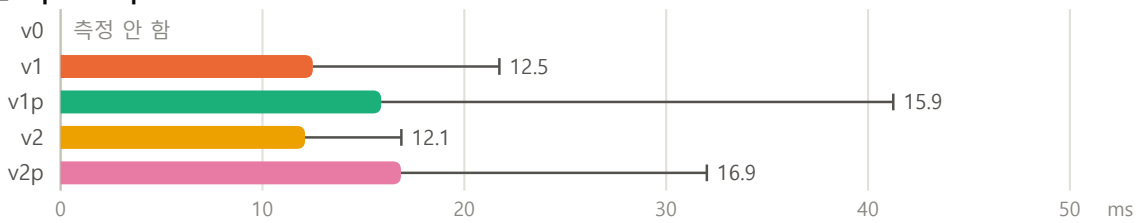
서버 processingMs · serverProcessingMs



서버 totalMs · serverTotalMs



작업 사이 빈 시간 · queueGapMs



막대 = p50(반복을 합친 표본; 값 지표는 반복의 중앙값), 가는 선 = p95, 숫자 = p50. 지표마다 축이 따로다.

14. real,prod · upload 전송 `real_upload_prod`

real,prod 프로파일: 사진 시그니처 검사, 작업마다 추론 서비스 healthz 확인. upload 전송. 워밍업 3 + 측정 20, 폴링 20 ms, 동시 접수 5건.

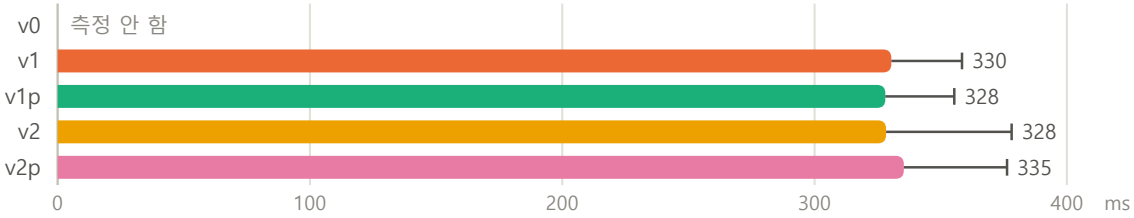
v0: 측정 안 함 — 이 버전에 없는 기능 (real_upload, prod)

지표	v0	v1	v1p	v2	v2p
전체 흐름 (POST→GLB 수신) `flowMs` · ms · p50 / p95	측정 안 함 이 버전에 없는 기능 (real_upload, prod)	330 / 358 n=60 기준	328 / 355 n=60 −0.7% = 차이 없음	328 / 378 n=60 −0.6% = 차이 없음	335 / 376 n=60 +1.5% = 차이 없음
완료 관측 (POST→COMPLETED) `completeMs` · ms · p50 / p95	측정 안 함 이 버전에 없는 기능 (real_upload, prod)	323 / 349 n=60 기준	320 / 347 n=60 −1.0% = 차이 없음	317 / 369 n=60 −1.9% = 차이 없음	327 / 369 n=60 +1.0% = 차이 없음
GLB 다운로드 `assetMs` · ms · p50 / p95	측정 안 함 이 버전에 없는 기능 (real_upload, prod)	8.20 / 10.7 n=60 기준	8.05 / 11.5 n=60 −1.7% = 차이 없음	9.28 / 13.7 n=60 +13.2% ! 악화	8.22 / 13.2 n=60 +0.4% = 차이 없음
업로드 요청 (POST→202) `postMs` · ms · p50 / p95	측정 안 함 이 버전에 없는 기능 (real_upload, prod)	42.1 / 53.7 n=60 기준	40.2 / 52.2 n=60 −4.5% = 차이 없음	278 / 305 n=60 +560.4% ! 악화	43.0 / 52.7 n=60 +2.1% = 차이 없음
서버 queuedMs `serverQueuedMs` · ms · p50 / p95	측정 안 함 이 버전에 없는 기능 (real_upload, prod)	19.0 / 30.1 n=60 기준	19.0 / 33.1 n=60 ±0.0% = 차이 없음	18.0 / 30.1 n=60 −5.3% = 차이 없음	23.0 / 40.0 n=60 +21.1% ! 악화
서버 processingMs `serverProcessingMs` · ms · p50 / p95	측정 안 함 이 버전에 없는 기능 (real_upload, prod)	257 / 281 n=60 기준	258 / 281 n=60 +0.6% = 차이 없음	253 / 293 n=60 −1.6% = 차이 없음	255 / 284 n=60 −0.8% = 차이 없음
서버 totalMs `serverTotalMs` · ms · p50 / p95	측정 안 함 이 버전에 없는 기능 (real_upload, prod)	279 / 299 n=60 기준	280 / 304 n=60 +0.4% = 차이 없음	273 / 312 n=60 −2.1% = 차이 없음	280 / 311 n=60 +0.4% = 차이 없음
작업 사이 빈 시간 `queueGapMs` · ms · p50 / p95	측정 안 함 이 버전에 없는 기능 (real_upload, prod)	17.8 / 23.6 n=12 기준	16.2 / 25.5 n=12 −8.9% = 차이 없음	17.6 / 23.1 n=12 −1.3% = 차이 없음	23.2 / 45.4 n=12 +30.5% = 차이 없음
기동 (Popen→healthz 200) `bench.startupMs` · ms · 값	측정 안 함 이 버전에 없는 기능 (real_upload, prod)	8,012 3회 기준	7,955 3회 −0.7% = 차이 없음	8,761 3회 +9.3% = 차이 없음	8,543 3회 +6.6% = 차이 없음
서버 CPU 점유 시간 (시나리오 + 뒤따른 백그라운드 일) `bench.serverCpuMs` · ms · 값	측정 안 함 이 버전에 없는 기능 (real_upload, prod)	7,641 3회 기준	8,297 3회 +8.6%	7,625 3회 −0.2%	10,609 3회 +38.9%

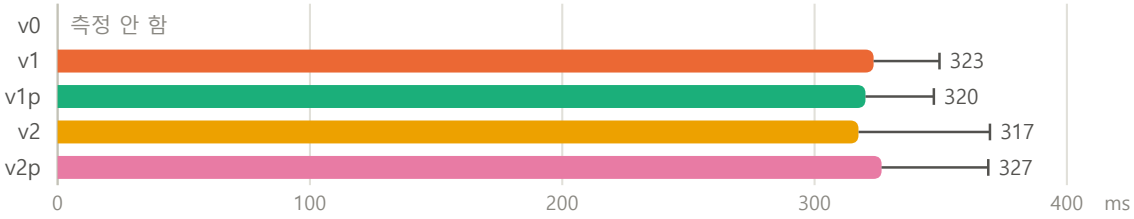
지표	v0	v1	v1p	v2	v2p
측정 전마다 서버가 쉴 때까지 더 기다린 합계 (확인 창 제외) `bench.settleTotalMs` · ms · 값	측정 안 함 이 버전에 없는 기능 (real_upload, prod)	2,513 3회	3,012 3회	3,010 3회	3,012 3회

v0 이전 (10-14, #1 머지) v1 현재 (4b 배포 기반) v1p 현재 + 개선 v2 다음 버전 후보 (v1.1) v2p 다음 버전 + 개선

전체 흐름 (POST→GLB 수신) · flowMs



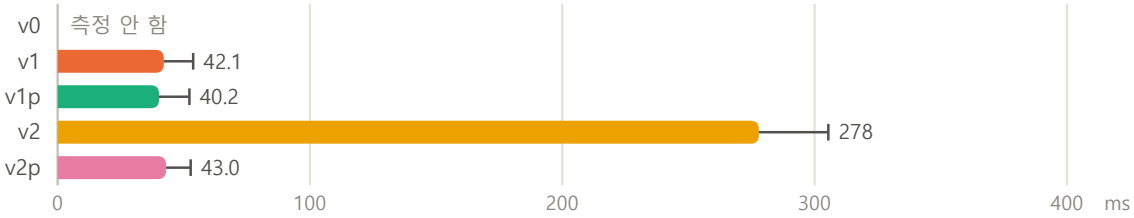
완료 관측 (POST→COMPLETED) · completeMs



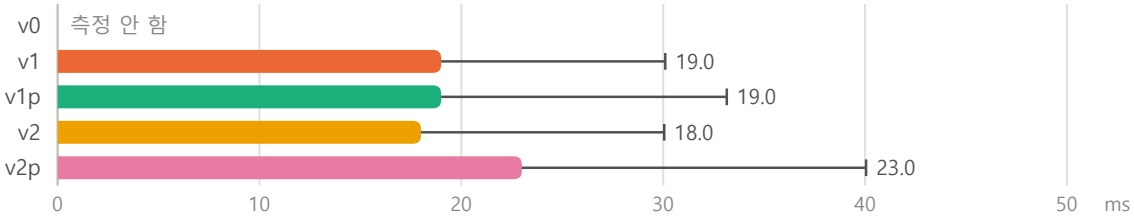
GLB 다운로드 · assetMs



업로드 요청 (POST→202) · postMs



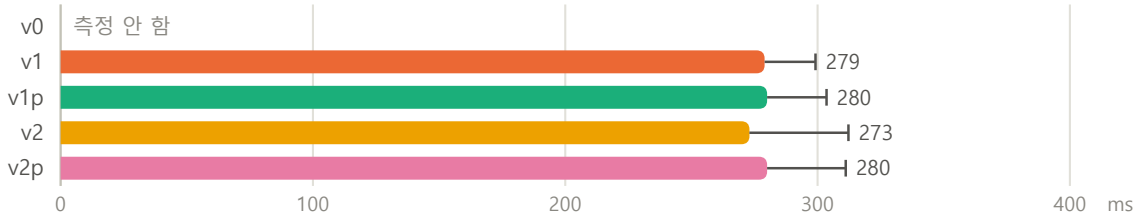
서버 queuedMs · serverQueuedMs



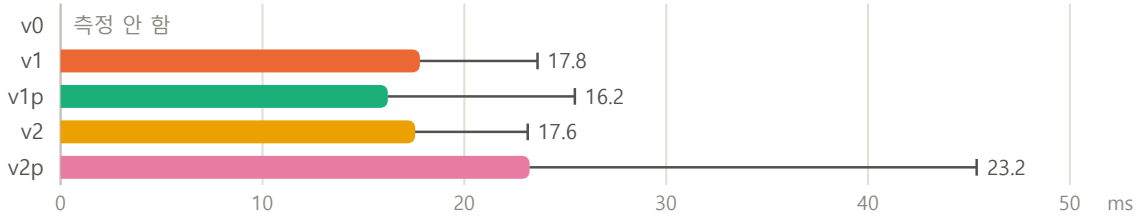
서버 processingMs · serverProcessingMs



서버 totalMs · serverTotalMs



작업 사이 빈 시간 · queueGapMs



막대 = p50(반복을 합친 표본; 값 지표는 반복의 중앙값), 가는 선 = p95, 숫자 = p50. 지표마다 축이 따로다.

개선 사례와 버전 비교

이 장의 숫자는 모두 이 보고서의 데이터(2026-10-31 14:34~15:30, 버전 5개 × 3회 교차, 같은 PC)에서 나온 **[실험 결과]** 이고, p50 은 세 반복을 합친 표본의 중앙값, 판정은 앞 장과 같은 규칙(|Δ| ≥ 10 % 이고 반복별 범위가 겹치지 않을 때만)이다. 원인 설명 가운데 확인하지 않은 것은 '가설'로 적었다.

한눈에

비교	지표	앞 → 뒤 (p50)	변화	판정
v1 → v1p (P1)	목록 첫 페이지, N=2000	13.1 → 3.6 ms	−72 %	개선
v1 → v1p (P1)	목록 중간 페이지, N=2000	22.1 → 15.0 ms	−32 %	개선
v1 → v1p (P1)	최대 RSS	499 → 449 MiB	−10 %	개선
v1 → v2 (기능 비용)	12 MP 사진 업로드	15.0 → 70.1 ms	+367 %	악화
v1 → v2 (기능 비용)	real·path 업로드 요청	43 → 272 ms	+532 %	악화
v1 → v2 (기능 비용)	폴링 처리량	1,752 → 1,291 요청/s	−26 %	악화
v2 → v2p (P4)	12 MP 사진 업로드	70.1 → 15.5 ms	−78 %	개선
v2 → v2p (P4)	real·path 전체 흐름	285 → 87 ms	−69 %	개선
v2 → v2p (P1)	목록 첫 페이지, N=2000	14.7 → 6.8 ms	−54 %	개선
v2 → v2p (P5)	PENDING 작업 조회	4.90 → 4.66 ms	−5 %	차이 없음

P1 작업 목록 키셋 페이지 (v1 → v1p)

- **실패 사례**: `GET /api/v1/jobs` 는 작업이 쌓일수록 느려질 구조였다. 요청마다 모든 작업을 읽어 정렬하고, 커서는 목록을 처음부터 훑어 찾았다.
- **원인 가설**: 비용이 페이지 크기가 아니라 전체 작업 수(N)에 비례한다. 같은 이유로 목록을 부를 때마다 모든 Job 엔티티가 메모리에 올라온다.
- **개선**: `(createdAt desc, id desc)` 키셋 조회에 limit+1, `jobs(created_at, id)` 인덱스. 커서는 그대로 마지막 항목의 id(계약의 '불투명 문자열' 유지, `JobListPagingTest`).
- **비교 실험**: N≈390·1000·2000 에서 첫 페이지(limit 20) 각 100회, 중간 페이지(limit 100) 각 50회.
- **결과**: 첫 페이지 p50 이 N≈390 에서 11.4 → 5.4 ms(−53 %), N=1000 에서 10.7 → 4.2 ms(−61 %), N=2000 에서 13.1 → 3.6 ms(−72 %). 중간 페이지는 N=1000 에서 −16 %, N=2000 에서 −32 %. 최대 RSS 가 499 → 449 MiB(−10 %)로 준 것도 목록마다 전체 작업을 올리지 않게 된 결과로 본다(가설). 단건 조회·업로드는 차이 없음.
- **부작용 관찰**: 지연 0 인 mock E2E 흐름(`e2e_mock`)이 18.9 → 23.5 ms(+24 %, 악화)로 나왔다. 서버 쪽 처리 시간(events.jsonl totalMs)은 3~4 → 4~5 ms 로 1 ms 단위의 차이이고, 흐름 시간의 차이는 대부분 10 ms 폴링 간격에 걸리는 횟수(작업당 1.38 → 1.46회)에서 온다. 인덱스가 늘어 쓰기가 조금 느려졌다는 가설은 1 ms 해상도로는 확인되지 않는다. 앱의 폴링 간격(1초)에서는 보이지 않는 크기다.

P4 썸네일을 202 뒤 백그라운드로 (v2 → v2p)

- **실패 사례**: 다음 버전 후보 v2 에서 업로드가 크게 느려졌다(v1 대비): 12 MP 사진 15 → 70 ms, 큰 사진 4장 136 → 366 ms, real 시나리오의 4.5 MB 사진 업로드 요청 43 → 272~278 ms, 12 MP 4스레드 동시 업로드 59 → 235 ms.
- **원인 가설**: 업로드 요청 안에서 첫 사진의 썸네일을 디코드했다. 디코드는 서버 전체에서 한 번에 하나라서 동시 업로드가 줄을 선다. 이 PC 에서는 실시간 백신이 갓 쓴 사진을 처음 열 때 검사해(4.5 MB 노이즈 PNG 첫 열기 216 ms, 다시 열면 1.3 ms) 그 대기도 요청에 들어간다.
- **개선**: 썸네일은 202 를 보낸 뒤 스레드 하나의 큐에서 만든다. `thumbnailUrl` 은 만들어지기 전까지 null(계약의 '없으면 null' 범위).
- **결과**: 12 MP 업로드 70.1 → 15.5 ms(−78 %), 동시 업로드 235 → 55 ms(−77 %), 큰 사진 366 → 139 ms(−62 %), real 업로드 요청 272~278 → 43~45 ms(−84 %), real·path 전체 흐름 285 → 87 ms(−69 %). 모두 v1 수준으로 돌아왔다.
- **같이 본 것**: 일은 사라진 것이 아니라 옮겨 갔다. 12 MP 시나리오의 서버 CPU 점유 시간은 v2 3.3 s, v2p 3.7 s 로 비슷하다(v1 은 1.7 s). 썸네일 큐가 뒤 측정에 섞이지 않도록 시나리오마다 서버가 쉴 때까지 기다린다(측정 방법). 그래도 흐름

안에서 백그라운드 디코드와 CPU 를 나누는 경우는 남는다: real 시나리오의 작업 사이 빈 시간(queueGapMs)이 6~18 → 11~23 ms 로 늘었다. 실제 GPU 추론은 수십 초라 무시할 크기다.

P5 대기 순번 인덱스 (v2 → v2p)

- **실패 사례**: v2 에서 PENDING 작업 조회가 v1 의 2.6 ms 에서 4.9 ms(+88 %)로 느려졌다(작업 약 2,030개가 쌓인 DB).
- **원인 가설**: 폴링마다 대기 순번을 세는 `countQueuedBefore` 가 작업 표 전체를 훑는다.
- **개선**: `jobs(status, queued_at, id)` 인덱스와 범위 조건, `job_runs(job_id)`·`(finished_at)`·`jobs(pet_id)` 인덱스.
- **결과**: 4.90 → 4.66 ms(−5 %, **차이 없음**). 가설이 맞지 않았다: N≈2,030 에서는 전체 훑기도 싸고, v1 대비 남은 +2 ms 는 다른 곳에 있다. 인덱스는 N 이 커질 때를 위한 것으로 남기되 효과는 이 측정으로 보이지 않았다고 적는다. 다음 확인은 아래 '남은 비용'.

이전 → 현재 (v0 → v1)

4b 배포 기반(prod 프로파일, 업로드 전송, GPU 감시·오프라인 대기, Discord 알림, H2 백업)을 더했지만 측정한 경로의 회귀는 없다: 기동, 메모리, 업로드, 동시 흐름, 폴링, 목록, real·path 모두 '차이 없음'. 지연 0 mock 흐름만 25.8 → 18.9 ms(−27 %, 개선)로 나왔는데 원인은 확인하지 않았다(작업당 폴링 1.52 → 1.38회).

다음 버전에 남은 비용 (v1 → v2p) [실험 결과] → 다음 개선 [계획]

v2p 가 기준 v1 보다 '악화'로 판정된 지표 전부다(나머지는 차이 없음, 목록 첫 페이지만 개선 −29~−48 %). 가설은 모두 확인하지 않았다.

지표	v1 → v2p (p50)	변화	가설	다음 확인
폴링 처리량	1,752 → 1,320 요청/s	−25 %	조회마다 대기 순번·예상 시간 계산과 썸네일 파일 확인	JFR 로 GET 한 번의 시간 분해, 끝난 작업은 계산 생략
PENDING 조회	2.6 → 4.7 ms	+79 %	예상 시간의 중앙값 질의(5초 캐시)와 응답 필드 증가	같은 프로파일링
단건 조회 (N≈390·1000·2000)	0.50~0.58 → 0.65~0.73 ms	+26~30 %	응답 필드(펫·썸네일·대기열) 매핑	같은 프로파일링
목록 중간 페이지 (limit 100)	N≈390 21.0 → 33.2, N=2000 22.1 → 25.2 ms	+14~58 %	항목마다 같은 매핑을 100번 — P1 이 줄인 조회 비용을 상쇄	같은 프로파일링, 목록 항목의 썸네일 확인을 한 번에
동시 E2E 처리량 (흐름 p50)	62.2 → 47.1 건/s (195 → 280 ms)	−24 % (+44 %)	위 조회 비용 + 백그라운드 썸네일	썸네일 없는 사진으로 분리 측정
작은 사진 업로드 (N≈390·1000 업로드)	13.2 → 16.9 ms (6.9 → 8.5, 5.9 → 8.1 ms)	+28 % (+24~38 %)	쉬지 않고 이어지는 업로드에서 앞 요청의 202 뒤 썸네일 일(디코드·파일 쓰기)이 겹침. 서버 uploadMs 도 6 → 8 ms. 사진마다 쉬고 재는 큰 사진·12 MP 업로드는 v1 수준	업로드 사이에 쉬고 다시 재기
지연 0 mock 흐름	18.9 → 30.2 ms	+60 %	같은 작업의 썸네일과 mock 처리가 겹침: 서버 totalMs p50 4 → 6 ms, 작업당 폴링 1.38 → 1.62회(10 ms 간격)	썸네일 없는 사진으로 분리 측정
기동 / 재기동	7.7 → 8.6 s / 7.9 → 8.9 s	+12 % / +13 %	빈(bean)·엔티티 증가	시작 로그의 구간 시간
real 작업 사이 빈 시간 (path / upload)	7.8 → 10.6 / 12.5 → 16.9 ms	+36 % / +35 %	백그라운드 썸네일과 CPU 공유(P4 '같이 본 것')	GPU 추론(수십 초)에 비해 작아 따로 하지 않음

측정에서 고친 것

- 대기 없이 잰 빠른 실행(커밋하지 않은 동작 확인)에서는 v2p 의 동시 E2E·폴링이 앞 시나리오의 썸네일 디코드와 CPU 를 다퉈 2~3배 느리게 나왔다. 시나리오마다 서버가 쉴 때까지 기다리고, 서버 CPU 점유 시간을 함께 남기도록 도구를 고친 뒤 이 데이터를 쟀다.

- 같은 PC 의 다른 프로그램(게임 클라이언트 등)이 고정한 CPU 를 차지해 서버가 멈추는 일이 있어 서버·스텁·부하 생성기를 보통 이상 우선순위로 띄웠다. 측정 도중 게임이 시작된 첫 실행(13:46)은 버리고, 게임이 끝나 3분 연속 조용해진 뒤 다시 쟀다. 이 데이터의 스위트 직전 배경 CPU 는 17~33 %(중앙 19 %).
- 기록해 둘 이상치: v1p 2회차의 real,prod 업로드 시나리오에서 워밍업 한 건이 약 3분 멈췄다(원인 미상). 측정 표본(흐름 20개)은 정상이지만 그 시나리오의 서버 CPU 점유 시간은 이상치다. 버린 첫 실행에서는 v2p 의 real 업로드 흐름 약 70개 중 1개가 COMPLETED 를 본 직후 에셋 409(PROCESSING)를 받았다. 이번 데이터(15 스위트)에서는 다시 나오지 않았다 — 원인 미상 **[계획]**.

하지 않은 개선과 이유

- 업로드 모드의 정리(`DELETE /artifacts`)를 추론 슬롯 밖으로(P2): 슬롯을 더 잡는 시간이 작업당 왕복 1회(루프백 수 ms, Tailscale 수십 ms)라 수십 초 걸리는 추론에 비해 작다. VM ↔ GPU PC 실측 뒤 다시 본다.
- 업로드 저장을 복사 대신 이동(P3): VM 의 Docker 에서는 Tomcat 임시 폴더와 저장소 볼륨이 다른 파일시스템이라 결국 복사다. 닫지 않던 입력 스트림은 코드 리뷰에서 고쳤다.
- GLB 의 ETag/304, 목록 기본 기간, 응답 압축: 상태 코드나 앱 동작이 바뀌어 계약 회의 안건이다.

다음 버전 측정 절차와 남은 측정

다음 버전(v3)을 만들었을 때

1. `docs/perf/versions.json` 의 `versions` 에 `v3` 를 더한다. `ref` 는 잴 커밋의 **SHA** 로 적는다(브랜치는 움직여서 커밋한 보고서를 다시 만들 수 없다). `capabilities` 에는 그 버전에 있는 기능(`mock`, `real_path`, `real_upload`, `prod`)만 넣는다.
2. `order` 끝에 `v3` 를 넣는다. 보고서의 색은 이 순서의 자리로 정해지므로 앞 라벨의 자리는 바꾸지 않는다.
3. `pairs` 에 직전 버전과의 쌍을 더한다. 예: `{"from": "v2p", "to": "v3", "title": "v1.2 의 변화"}`. 배포 버전이 바뀌면 `baseline` 도 옮긴다.
4. 기준 버전·직전 버전과 **같은 실행에서** 잰다. 날짜가 다른 실행의 결과를 섞어 판정하지 않는다: 다른 프로그램의 부하, 온도, OS·JDK 업데이트가 실행마다 달라 판정 규칙(반복 사이 범위)이 그 차이를 걸러 내지 못한다.

```
python scripts/perf/run.py --versions v1,v2p,v3 --reps 3 --versions-file docs/perf/versions.json --stub-python <fastapi·uvicorn 이 깔린 python>
python scripts/perf/report.py --data docs/perf/data/<새 폴더> --versions-file docs/perf/versions.json --notes docs/perf/notes --out docs/perf/backend-perf-report.pdf
```

5. 개선이 있으면 '개선 사례'와 같은 순서(실패 사례 → 원인 가설 → 개선 → 비교 실험)로 `docs/perf/notes/` 에 적는다. 새 데이터 폴더, PDF, 노트를 한 커밋에 올린다.

측정 전 점검

- 무거운 프로그램(게임 클라이언트, 화상 회의, 다른 빌드)을 끈다. 이번 측정은 끄지 못한 채 진행했다. 그 부하는 보고서 '한계'에 스위트마다 잰 CPU 사용률로 적혀 있다.
- 노트북이면 전원을 연결하고 절전 모드를 끈다.
- `--quick --reps 1` 로 한 번 돌려 모든 시나리오가 `ok` 인지 먼저 본다(약 1분 × 버전 수).

아직 재지 않은 것 [계획]

항목	왜 필요한가	방법	담당
클라우드 VM(2 vCPU)에서 같은 시나리오	이 보고서는 개발 PC 에서 서버를 논리 CPU 2개(물리 코어 1개)에 고정해 VM 을 흉내 낸 값이다	VM 에 JDK 17·Python 을 두고 같은 도구를 실행한다. 개발 PC 에서 VM 으로 원격 부하를 거는 것은 도구 확장이 필요하다	Backend
VM ↔ GPU PC(Tailscale) 업로드 모드의 전송 시간	사진 업로드, GLB 다운로드, `DELETE /artifacts` 가 루프백이 아니라 실제 망에서 얼마인지	VM 의 Spring(`real,prod`, 업로드 모드)과 GPU PC 의 추론 서비스로 E2E. events.jsonl 의 `processingMs` 와 `inferMs`·`convertMs` 의 차이가 전송·저장 시간이다	Backend + 3D
실제 AnimalLift 추론 시간과 GPU 메모리	이 보고서의 real 시나리오는 모델 없는 패스스루 스텁이라 추론 시간이 0 이다	exp-01 (`generation/experiments/`)	3D
Unity 로딩 시간과 FPS	서버 수치만으로는 사용자가 기다리는 시간을 다 설명하지 못한다	앱의 metrics.csv (METRICS §3·§4)	Unity

부록

데이터 파일

- `docs/perf/data/2026-10-31_1434`: run.json, v0-r1.json, v0-r2.json, v0-r3.json, v1-r1.json, v1-r2.json, v1-r3.json, v1p-r1.json, v1p-r2.json, v1p-r3.json, v2-r1.json, v2-r2.json, v2-r3.json, v2p-r1.json, v2p-r2.json, v2p-r3.json

명령

- 측정: `python scripts/perf/run.py --versions v0,v1,v1p,v2,v2p --reps 3 --versions-file docs/perf/versions.json --stub-python C:/Users/minwo/AppData/Local/Temp/claude/C--Users-minwo-OneDrive-------capstone-Beside/7b66930a-7eb1-4f94-991f-a3a35ae2caf2/scratchpad/venv/Scripts/python.exe` (2026-10-31T14:34:05+09:00 ~ 2026-10-31T15:30:28+09:00, 상태 ok)
- 보고서: `python scripts/perf/report.py --data docs/perf/data/2026-10-31_1434 --versions-file docs/perf/versions.json --notes docs/perf/notes --out docs/perf/backend-perf-report.pdf`

도구와 버전 커밋

- 측정 도구 커밋 `ed37f406b0b50726f704844e6027831f46d2538c`
- 보고서 생성 도구 커밋 `8ecbd8f6e821e3bb9df36e3d8735a9a78043a3c9`
- v0: `e0e1c2b34b1b94669d824fd0fd4afeea9fb800f1` (e0e1c2b34b1b94669d824fd0fd4afeea9fb800f1)
- v1: `af4a747b1c36f2380d1ff69a523b5a9dbaca7503` (af4a747b1c36f2380d1ff69a523b5a9dbaca7503)
- v1p: `033f3f3719745e3d4776c5ea6e4e2e7db536f027` (033f3f3719745e3d4776c5ea6e4e2e7db536f027)
- v2: `6a3959f60380947ad2f430be24f5d50661a57c95` (6a3959f60380947ad2f430be24f5d50661a57c95)
- v2p: `ecc5fb5f9ba0bee443137ce8c59a5223174e7dc2` (ecc5fb5f9ba0bee443137ce8c59a5223174e7dc2)

생성 2026-10-31T15:59:26+09:00